



武汉大学
WUHAN UNIVERSITY

S-OS (SuperOS)

内核设计文档

成员：包一帆，李嘉乐，庄廷钊

指导教师：蔡朝晖

2026 年 8 月

目录

第一章 概述	2
1.1 系统架构总图	2
1.2 项目定位	3
1.3 代码组织	3
第二章 进程与线程管理	4
2.1 任务模型：三层结构	4
2.1.1 proc_t: 资源容器的组织	4
2.1.2 thread_t: 执行现场的组织	5
2.1.3 cpu_t: 最小化调度现场	5
2.2 进程创建路径对比	6
2.2.1 fork() 的完整复制路径	6
2.2.2 clone() 的完整复制路径	6
2.2.3 clone_thread() 的共享页表机制	7
2.3 调度器与上下文切换	7
2.3.1 scheduler(): 二级扫描逻辑	7
2.3.2 时间片轮转与超时检测	8
2.4 阻塞、唤醒与同步路径	9
2.4.1 基于 channel 的睡眠模型	9
2.4.2 Futex: 线程级等待与唤醒	9
2.5 信号与定时器	11
2.5.1 信号状态的组织	11
2.5.2 信号发送与终止标记的协作	11
2.6 退出、等待与资源回收	11
2.6.1 exit -> reparent -> ZOMBIE -> wait -> freeproc 回收链	11
2.6.2 freeproc() 的资源释放顺序	12
2.6.3 initproc 对孤儿进程的回收	12
2.6.4 线程退出与延迟回收	13
2.7 本章小结	13
第三章 内存管理	14
3.1 总体架构	14
3.2 Buddy System	14
3.2.1 数据结构	14

3.2.2	buddy_init(): 自举过程	15
3.2.3	buddy_alloc(): 分配算法	15
3.2.4	buddy_free(): 释放与合并算法	16
3.2.5	PAGE_NUM 的双架构差异	17
3.2.6	页级引用计数	17
3.3	SLAB 分配器	17
3.3.1	固定 size class 设计	17
3.3.2	slab_alloc(): 分配流程	18
3.3.3	自举过程: simple_alloc -> slab_init	18
3.4	页表与地址映射	18
3.4.1	walk(): 多级页表遍历核心算法	18
3.4.2	RISC-V 与 LoongArch 的页表差异	19
3.5	VMA 与用户地址空间	20
3.5.1	地址空间布局	20
3.5.2	VMA 链表与区域语义	20
3.5.3	alloc_vma / alloc_mmap_vma 的地址选择策略	21
3.5.4	mmap / munmap 的实现流程	21
3.5.5	VMA 并发保护与文件映射提交	21
3.5.6	fork() 时 VMA 的共享与私有处理	22
3.5.7	pagefault_handler 的决策流程	22
3.6	本章小结	23
第四章	硬件抽象层与异常处理机制	24
4.1	分层架构与代码边界	24
4.2	双架构启动链	25
4.2.1	RISC-V 启动链	25
4.2.2	LoongArch 启动链	25
4.3	Trapframe 与 Context 的分工	26
4.4	用户态 Trap 主路径	27
4.4.1	usertrap() 异常分类	28
4.4.2	devintr() 中断分发	29
4.4.3	平台定时器与诊断输出	29
4.4.4	缺页处理	29
4.5	内核态 Trap	30
4.6	LoongArch TLB Refill	30
4.7	hsai_usertrapret() 返回路径	32
4.8	本章小结	33

第五章 系统调用与用户态接口	34
5.1 设计目标	34
5.2 ABI 入口与用户态调用桩	35
5.2.1 双架构调用约定	35
5.2.2 GEN_USER_SYSCALL 宏	35
5.3 从 Trap 到 syscall() 的统一分发	35
5.3.1 syscall() 分发器	35
5.4 参数搬运机制	37
5.5 Linux 用户态 ABI 兼容边界	37
5.5.1 clone 参数归一化	38
5.5.2 TCGETS: 终端结构的定长兼容	38
5.6 系统调用分类	39
5.7 兼容层设计	40
5.7.1 用户态运行时视图与虚拟文件接口	40
5.7.2 动态 maps 与真实执行路径	41
5.7.3 LD_LIBRARY_PATH 注入	41
5.7.4 /dev/shm 重映射	42
5.7.5 Synthetic Mount 机制	42
5.8 本章小结	43
第六章 文件系统与程序装载	44
6.1 总体架构	44
6.2 文件对象层	45
6.2.1 struct file 的类型系统	45
6.2.2 生命周期管理	46
6.2.3 读写类型分发	46
6.2.4 FILE_OPS 分发表	46
6.3 EXT4 适配层	47
6.3.1 打开路径: vfs_ext4_openat	47
6.3.2 读写路径: 分页中转缓冲区	47
6.3.3 EXT4 大文件与稀疏文件读取	47
6.3.4 目录遍历	48
6.3.5 元数据访问	48
6.3.6 删除操作中的 inode 引用协调	48
6.4 块缓存层	48
6.4.1 核心数据结构	48
6.4.2 四个核心接口	49
6.4.3 bget 的 LRU 回收策略	49

6.4.4	blockdev.c 的 EXT4 适配	49
6.5	管道实现	50
6.6	BusyBox 虚拟文件	50
6.6.1	虚拟文件列表	50
6.6.2	按路径分发	51
6.7	execve 装载器	51
6.7.1	执行路径与软链接解析	51
6.7.2	执行流程	52
6.7.3	PT_LOAD 多段统一装载	52
6.7.4	PIE 支持	53
6.7.5	动态解释器处理	53
6.7.6	辅助向量构造	53
6.7.7	脚本兼容与重定向	54
6.8	本章小结	54
第七章	驱动层设计与实现	55
7.1	概述	55
7.2	控制台子系统	55
7.2.1	读写分离架构	55
7.2.2	行规约处理	56
7.2.3	UART 发送环形缓冲	56
7.2.4	字符设备注册	56
7.3	定时器	57
7.3.1	双架构定时器初始化对比	57
7.3.2	事件链: timer_tick 到 SIGALRM	57
7.4	RISC-V PLIC	58
7.5	RISC-V VirtIO MMIO 块设备	59
7.5.1	virtio_disk_init 状态机	59
7.5.2	三描述符链与请求提交	60
7.5.3	中断回收	60
7.5.4	描述符表内存布局	60
7.6	LoongArch PCI 与 VirtIO-PCI 块设备	61
7.6.1	DMA 缓冲区与非缓存访问	61
7.6.2	PCI 设备枚举	62
7.6.3	Capability 解析	62
7.6.4	轮询完成路径	63
7.6.5	两种架构的差异对比	63
7.7	驱动与上层接口	64

目录	5
7.8 本章小结	64
第八章 总结与展望	65
8.1 设计总结	65
8.2 后续优化方向	65

第一章 概述

1.1 系统架构总图

系统采用自底向上的分层结构，层间依赖关系如图 1.1 所示。SOS 内核在统一代码基线上同时支持 RISC-V 64 和 LoongArch 64 两种体系结构，图中以体系结构相关与体系结构无关两类模块展示了这一设计。

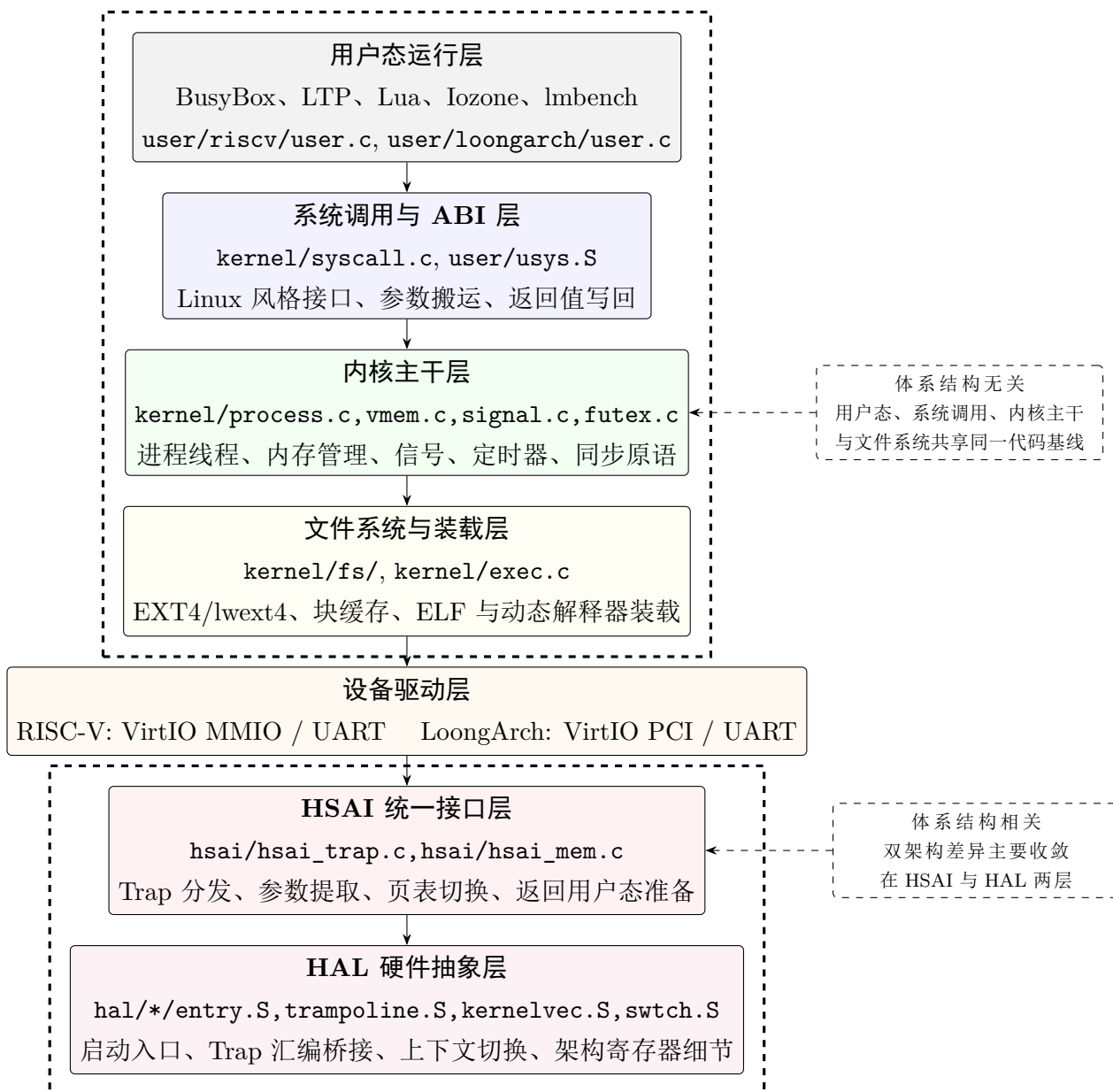


图 1.1: SOS 内核全架构分层图

1.2 项目定位

SOS (SuperOS) 是一个由 C 语言编写的宏内核操作系统，面向 RISC-V 64 和 LoongArch 64 两种体系结构，覆盖进程与线程管理、虚拟内存管理（Buddy + Slab + 页表 + VMA）、EXT4 文件系统（集成 lwext4）、VirtIO 块设备驱动（MMIO 与 PCI）以及 Linux 风格系统调用接口（进程、内存、文件、定时器、事件、socket 类别），在初赛适配了 basic、buxybox、libctest、libcbench、lua、部分 ltp 等测试用例，在决赛一阶段通过 cagent 全部测例和 buildstorm 环境检查与最小构建，最终尝试下板。

1.3 代码组织

各目录的职责和代表性文件见表 1.1。

表 1.1: 目录结构与职责

目录	职责	代表性文件
hal/	架构入口、寄存器现场保存恢复、上下文切换、UART	entry.S, trampoline.S, swtch.S, uart.c
hsai/	Trap 分发、页表硬件配置、PLIC、内核打印	hsai_trap.c, hsai_mem.c, plic.c, print.c
kernel/	进程线程、内存管理、系统调用、信号、定时器	process.c, vmem.c, syscall.c, signal.c, futex.c
kernel/fs/	文件对象、路径处理、块缓存、EXT4 适配	vfs_ext4.c, file.c, bio.c, blockdev.c
kernel/driver/	块设备驱动（VirtIO MMIO / PCI）	virt.c, pci.c, virtio_disk.c
kernel/exec.c	ELF 与脚本装载	exec.c
user/	首个用户态程序、系统调用桩、测试框架	user.c, usys.S, usys_macro.inc
include/	内核核心头文件、HAL/HSAI 接口、文件系统定义	process.h, thread.h, vmem.h, vma.h

架构依赖关系为：hal/ → hsai/ → kernel/（核心层与文件系统层）。其中，hal/ 和 hsai/ 主要容纳架构相关代码，kernel/ 主体保持双架构共享。驱动层在 kernel/driver/ 内部按架构分目录，例如 kernel/driver/riscv/ 和 kernel/driver/loongarch/。

第二章 进程与线程管理

2.1 任务模型：三层结构

当前内核将执行实体拆分为三个层次：进程（`proc_t`）持有资源，线程（`thread_t`）持有执行现场，CPU 本地状态（`cpu_t`）保存调度器切换现场。三层结构的核心动机是：资源不随执行流切换而复制，执行现场不因进程创建而膨胀，调度器不关心上层语义。决赛中线程还拥有独立的信号屏蔽字、待处理信号和 `clear_child_tid`，因此线程的生命周期和异步事件可以独立于进程资源运行。

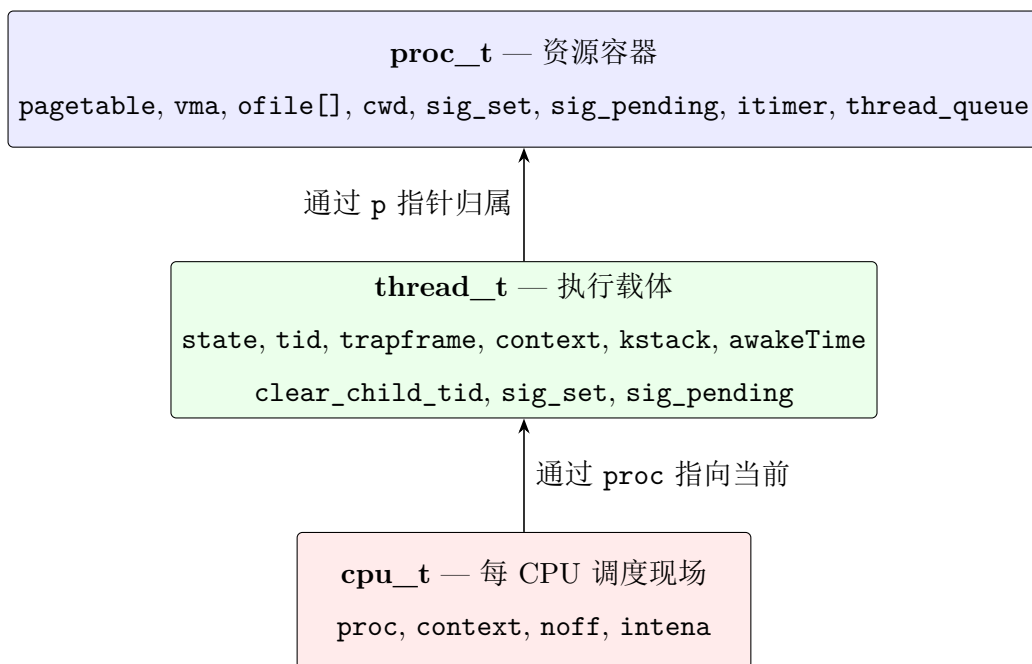


图 2.1: 进程/线程/CPU 的三层关系

2.1.1 `proc_t`: 资源容器的组织

`proc_t` 定义在 `include/kernel/process.h`。其成员按职能可分为五组：

职能组	关键字段	用途
地址空间	<code>pagetable, vma, sz, virt_addr</code>	持有用户页表指针、VMA 链表头、低地址连续区大小和程序入口偏移
文件系统	<code>ofile[NOFILE], cwd, ofn</code>	打开文件描述符表、当前工作目录、文件描述符数量限制
信号	<code>sig_set, sigaction[], sig_pending</code>	信号阻塞掩码、每个信号的处理函数、挂起信号位图

定时器	<code>itimer,</code>	区间定时器值 (<code>setitimer</code> 接口)、时钟到期计数、定时器激活标记
	<code>alarm_ticks,</code>	
	<code>timer_active</code>	
线程管理	<code>thread_queue,</code>	线程链表头、“当前线程”指针、线程计数
	<code>main_thread,</code>	
	<code>thread_num</code>	

表 2.2: `proc_t` 关键成员分组

上表只列出进程级共享状态；线程级的 `sig_set`、`sig_pending` 和 `clear_child_tid` 由各个 `thread_t` 分别维护。

2.1.2 `thread_t`: 执行现场的组织

`thread_t` 定义在 `include/kernel/thread.h`，集中保存执行流自身的状态：

- **调度状态**: `state` (枚举 `t_UNUSED` `t_TIMING`)、`chan` (等待 channel)、`awakeTime` (定时唤醒截止时间)。
- **执行现场**: `trapframe` (Trap 返回用寄存器镜像)、`context` (调度切换用 callee-saved 寄存器)。两者分工明确——`trapframe` 在用户态与内核态之间切换时保存/恢复全部寄存器，`context` 仅保存 `ra/sp/s0-s11`。
- **内核栈**: `kstack` 与 `kstack_pa`。主线程复用进程的 `kstack`，附加线程由 `clone_thread()` 单独分配一页物理页作为内核栈，并映射到内核页表。
- **链表节点**: `elem` 用于挂入 `proc_t` 的 `thread_queue`，形成进程内的线程集合。
- **线程运行时状态**: `clear_child_tid` 用于退出时清零用户态 `tid` 并唤醒 `pthread_join()`；`sig_set` 和 `sig_pending` 用于保存当前线程的信号屏蔽字和待处理信号。

2.1.3 `cpu_t`: 最小化调度现场

`cpu_t` 定义在 `include/kernel/cpu.h`，按 CPU 保存当前进程、调度上下文和中断嵌套状态。当前调度主路径仍按单 CPU 调度组织，但 `cpu_t` 已明确划分 CPU 状态与进程状态的边界；多 hart 启动基础不等同于完整的 SMP 调度。切换时 `hsai_swch()` 仅在 `cpu->context` 与 `p->context` 之间切换，不涉及地址空间或文件系统等高层状态。

2.2 进程创建路径对比

维度	fork()	clone()	clone_thread()
页表策略	uvmcopy() 逐页复制内容	uvmcopy() 逐页复制内容	CLONE_VM 下共享父进程页表（无复制）
VMA 策略	vma_copy() + vma_map() 深拷贝（私有页复制，共享页引用计数）父进程返回子进程	同 fork()	继承父进程 thread_queue，不复制 VMA
返回值	PID，子进程返回 0	子进程 PID	新线程 TID
TLS 处理	不涉及	不涉及	CLONE_SETTLS 设置 tp 寄存器用户提供
栈布局	复制父进程栈	用户提供 stack + args_t	stack_va + 新内核栈
文件描述符	dup() 全部打开文件	dup() 全部打开文件	共享（不复制）

表 2.4: 三种创建路径的对比

2.2.1 fork() 的完整复制路径

fork() 对应“完整复制进程”的语义：新建子进程对象，复制父进程的大部分可见运行状态，同时给予子进程分配独立的地址空间与主线程现场。其关键步骤可概括为：

1. 调用 allocproc() 创建 proc_t，完成 PID、内核栈、活动 trapframe 等基础资源初始化。
2. 调用 uvmcopy() 复制父进程用户页，为子进程建立独立页表与物理页副本。
3. 调用 vma_copy() 复制 VMA，共享映射保留共享关系，私有映射维持父子地址空间分离。
4. 通过 dup() 复制文件描述符，继承工作目录、身份权限、信号与定时器等进程属性。
5. 复制执行现场至子进程 trapframe，子进程返回值置 0，父进程返回子 PID。
6. 子进程设为可运行并入调度队列，父子调度独立，仅共享显式定义的内核对象。

2.2.2 clone() 的完整复制路径

sys_clone() (kernel/syscall.c) 承担创建语义的总分发：

```
sys_clone(flags, stack, ptid, tls, ctid):
    if stack == 0 -> fork() // 完整复制进程
```

```
if flags & CLONE_VM -> clone_thread() // 共享地址空间, 创建线程
else -> clone() // 独立页表, 新进程
```

2.2.3 clone_thread() 的共享页表机制

clone_thread() (kernel/process.c) 不调用 uvmcopy(), 新线程直接使用父进程的 p->pagetable。其步骤为:

1. 调用 alloc_thread() 分配 thread_t, 该线程的 trapframe 是新分配的物理页, 内容通过 copytrapframe() 从父进程 p->trapframe 复制。
2. 分配一页物理页作为新线程的内核栈, 并通过 mappages() 映射到内核页表 (虚拟地址为 p->kstack - PGSIZE * p->thread_num)。
3. 从用户空间复制 args_t (含入口函数地址、参数、控制信息), 设置新线程的 trapframe->epc/era 为用户入口, sp 为用户提供的栈指针。
4. 处理 CLONE_PARENT_SETTID、CLONE_CHILD_SETTID、CLONE_CHILD_CLEARTID 等标志, 将线程 ID 写入指定用户地址。
5. 设置 t->state = t_RUNNABLE, 挂入父进程的 thread_queue。

线程创建路径已经形成从独立现场构造、TLS 设置、tid 地址处理到线程队列入队的完整路径。跨架构用户态 clone 参数顺序在第五章 ABI 边界统一处理, 本节只描述架构无关的线程创建语义。

2.3 调度器与上下文切换

2.3.1 scheduler(): 二级扫描逻辑

调度器主循环实现在 kernel/process.c, 扫描顺序为先选进程, 再选线程。

Algorithm 2.1: scheduler() 主循环算法

```
1: while true do
2:   for all 进程池中的进程 p do
3:     if p 未处于 RUNNABLE then
4:       continue
5:     end if
6:     for all p 的 thread_queue 中的线程 t do
7:       if t 处于 t_RUNNABLE then
8:         选择 t 作为本轮执行线程
9:       else if t 处于 t_TIMING 且 awakeTime 已到期 then
10:        将 t 视为可运行线程并参与本轮调度
11:       else
12:         continue
13:       end if
```

```

14:          调用 futex_clear(p->main_thread) 清理残留等待项
15:          令 p->main_thread = t
16:          复制线程私有 context 和 trapframe 到进程活动镜像
17:          设置 p->state = RUNNING, t->state = t_RUNNING
18:          调用 hsai_swch() 进入该进程
19:          从 hsai_swch() 返回后保存线程现场
20:          if t 未成为 t_ZOMBIE then
21:              将 t 移至线程队列队尾
22:          end if
23:          break
24:      end for
25:      释放 p 的锁并继续扫描下一进程
26:  end for
27: end while

```

线程可运行性的判定包括两个条件：

- 显式处于 t_RUNNABLE 状态；或
- 处于 t_TIMING 状态且 awakeTime <= r_time(), 只在绝对到期时间到达后重新进入调度。

选中线程后的现场切换涉及三次复制：

1. p->main_thread = t: 将进程的”当前线程”指针指向被选中线程。
2. copycontext(&p->context, &p->main_thread->context): 将线程私有上下文复制到进程活动镜像。
3. copytrapframe(p->trapframe, p->main_thread->trapframe): 将线程私有 Trap 帧复制到进程活动镜像。

切换前还调用 futex_clear(p->main_thread)，清理该线程在 futex 等待队列中的残留项。

线程状态为 t_ZOMBIE 时不会再次插入线程队列。调度器从 hsai_swch() 返回后，确认当前线程已经离开其执行现场，再调用线程回收路径；普通线程则重新移动到线程队列尾部。这样线程退出路径不会在仍使用自身内核栈时释放该栈。

除僵尸线程外，处于 t_UNUSED 的线程也不会重新入队；调度器会跳过这两类已经不能执行的线程。

2.3.2 时间片轮转与超时检测

当前调度器不维护单独的每线程时间片计数器。用户态时钟中断在 hsai/hsai_trap.c 中被捕获后，调用 yield() 退回调度器，同时调用 timer_tick() 更新全局 ticks 并遍历进程池检查定时器到期。超时线程唤醒由 scheduler() 中 t_TIMING 判定逻辑承担：futex 等待中设置了 awakeTime 的线程，在调度器扫描时被自动发现并改为可运行。

2.4 阻塞、唤醒与同步路径

2.4.1 基于 channel 的睡眠模型

`sleep_on_chan()` 与 `wakeup()` 构成了内核内部最常用的阻塞原语。

`sleep_on_chan()` 的核心逻辑：

```
sleep_on_chan(chan, lk):
    if lk != &p->lock:
        acquire(&p->lock) // 防止错过 wakeup
        release(lk) // 释放调用者的锁
    p->chan = chan
    p->state = SLEEPING
    p->main_thread->state = t_RUNNABLE // 线程保持在就绪态
    sched()
    p->chan = 0
    if lk != &p->lock:
        release(&p->lock)
        acquire(lk)
```

关键设计决策：将 `p->main_thread->state` 设置为 `t_RUNNABLE` 而非 `t_SLEEPING`。这意味着 channel 睡眠机制本质上是**进程级阻塞**——进程状态变为 `SLEEPING`，但线程保留就绪状态。这是因为当前实现中每个进程在 channel 睡眠时只存在一个活跃线程，没有必要将线程也标记为阻塞。

`wakeup()` 与之对应，只扫描进程池：

```
wakeup(chan):
    for p in pool:
        if p->state == SLEEPING && p->chan == chan:
            p->state = RUNNABLE
```

2.4.2 Futex: 线程级等待与唤醒

Futex 子系统实现在 `kernel/futex.c`。其核心数据结构是一个线性数组：

```
typedef struct futex_queue {
    uint64 addr; // futex 用户空间地址
    thread_t *thread; // 在该地址上等待的线程
    uint8 valid; // 队列项是否有效
} futex_queue_t;

futex_queue_t futex_queue[FUTEX_COUNT]; // FUTEX_COUNT = 2048
```

`futex_wait()` 的查找与等待流程：

Algorithm 2.2: futex_wait() 队列查找算法

```

1: 在 futex_queue[0..FUTEX_COUNT) 中查找首个 valid == 0 的空项
2: if 未找到空项 then
3:   panic
4: end if
5: 在该队列项中写入 addr、等待线程指针和 valid = 1
6: if ts 为 NULL then
7:   设置 th->state = t_SLEEPING
8: else
9:   计算  timeout = ts->tv_sec * CLK_FREQ + ts->tv_nsec * CLK_FREQ /
      1000000000ULL
10:   设置 th->awakeTime = r_time() + timeout
11:   设置 th->state = t_TIMING
12: end if
13: 设置 th->p->state = RUNNABLE
14: 调用 sched() 主动让出 CPU

```

与 channel 睡眠的关键区别：futex 等待将线程状态置为不可运行（t_SLEEPING 或 t_TIMING），同时将进程状态设回 RUNNABLE。这使得调度器可以在该进程的其他线程之间切换。

futex_wake() 扫描队列，将匹配地址的有效项对应线程改回 t_RUNNABLE，并同步将所属进程置为 RUNNABLE；当唤醒数量为负数时表示广播。futex_requeue() 先按唤醒数量唤醒等待者，再将剩余等待者的地址改为新的 futex 地址，用于条件变量和互斥锁之间的等待者迁移。futex_clear() 在线程被调度器选中或退出时调用，用于清除该线程在 futex 等待队列中的残留项。

pthread 同步路径可以概括为：

```

futex_wait:
  保存用户地址和线程等待项
  设置 t_SLEEPING 或 t_TIMING
  保持所属进程 RUNNABLE
  sched()
  由 wake、requeue、超时或信号重新进入调度
  返回，或以 EINTR 结束阻塞调用

```

futex、nanosleep() 和 poll() 返回前检查未屏蔽的待处理信号：futex 路径检查当前线程的信号状态，nanosleep() 和 poll() 路径同时合并进程级与线程级状态。线程定向信号会唤醒目标线程；阻塞调用被信号打断时返回 EINTR。调度器只在 awakeTime <= r_time() 时选择定时等待线程，避免把不同时间单位的超时值直接混用。

2.5 信号与定时器

2.5.1 信号状态的组织

信号状态按作用范围分为进程级和线程级两部分。`proc_t` 保存进程共享的处理动作、挂起信号和终止状态，`thread_t` 保存当前执行线程的屏蔽字和待处理信号：

- `sigaction[SIGRTMAX+1]`：处理函数配置，通过 `sys_rt_sigaction()` 设置。
- `sig_set`：信号阻塞掩码，通过 `sigprocmask()` 修改。实现中强制 `SIGKILL`、`SIGTERM` 和 `SIGSTOP` 不可被阻塞（`kernel/signal.c`）。
- `sig_pending`：进程级挂起信号位图，由 `kill()` 和 `timer_tick()` 写入。
- `killed`：终止标记，当终止类信号未安装处理函数时同步设置。
- `thread_t.sig_set` 与 `thread_t.sig_pending`：当前线程的信号屏蔽字和线程定向待处理信号。

2.5.2 信号发送与终止标记的协作

`kill()` 的执行逻辑：

```
kill(pid, sig):
    for p in pool:
        if p->pid == pid:
            p->sig_pending.__val[0] |= (1UL << sig)
            if signal_should_terminate(sig) AND
                p->sigaction[sig].sa_handler == NULL AND
                (p->killed == 0 || p->killed > sig):
                p->killed = sig
            if p->state == SLEEPING:
                p->state = RUNNABLE
```

`tgkill()` 和 `tkill()` 按 `tid` 在 `thread_queue` 中定位目标线程，并将信号写入目标线程的 `sig_pending`；目标线程处于 `futex`、`nanosleep()` 或 `poll()` 等阻塞状态时，内核将其重新置为可运行。`rt_sigprocmask()` 修改当前线程的屏蔽字，新线程继承创建线程的屏蔽状态。

Trap 层对进程级 `killed` 和线程级待处理信号的消费发生在阻塞系统调用和 `usertrap()`（`hsai/hsai_trap.c`）路径中。完整的用户态 `signal frame` 构造和处理函数返回路径仍属于后续完善方向。

2.6 退出、等待与资源回收

2.6.1 `exit` → `reparent` → `ZOMBIE` → `wait` → `freeproc` 回收链

资源回收链以 `exit()` 为起点：

1. 禁止 `initproc` 退出。
2. 通过 `get_file_ops()->close()` 关闭全部打开文件。

3. 获取全局 `parent_lock`，调用 `reparent(p)` 将子进程过继给 `initproc`。对于已是 ZOMBIE 的子进程直接调用 `freeproc()` 回收。
4. 唤醒父进程 `wakeup(p->parent)`。
5. 设置 `p->state = ZOMBIE`, `p->main_thread->state = t_ZOMBIE`。
6. 通过 `sched()` 永久离开运行态。

父进程通过 `wait()` 承担回收职责。`wait()` 扫描进程池查找属于当前进程的 ZOMBIE 子进程，取出退出状态后调用 `freeproc()` 真正回收；若未找到且未设置 `WNOHANG`，则通过 `sleep_on_chan(p, &p->lock)` 等待子进程退出。

2.6.2 `freeproc()` 的资源释放顺序

`freeproc()` 只在整个进程进入 ZOMBIE 并由父进程回收时执行，负责进程对象的最终释放；已退出的附加线程由调度器返回路径中的 `reap_thread()` 单独回收。其释放顺序严格按照“进程对象 → 线程队列 → 地址空间 → 文件对象”展开：

Algorithm 2.3: `freeproc()` 的资源释放算法

```

1: 调用 pmem_free_pages(p->trapframe, 1) 释放进程活动 trapframe
2: for all 线程队列中的线程 t do
3:   if t 的 trapframe 与进程活动 trapframe 不同 then
4:     释放线程私有 trapframe
5:   end if
6:   if t 是附加线程 then
7:     解除附加线程内核栈映射并释放 kstack_pa
8:   end if
9:   设置 t->state = t_UNUSED，并将其放回 free_thread
10: end for
11: 调用 free_vma_list(p) 逐个释放 VMA
12: 调用 proc_freepagetable(p, p->sz) 释放用户页和多级页表
13: 对全部已打开文件逐描述符调用 get_file_ops()->close()
14: 重置进程槽位，例如 pid = 0、state = UNUSED、main_thread = NULL

```

关键细节：通过 `t->trapframe != p->trapframe` 判断避免重复释放主线程的 `trapframe`。附加线程在创建时分配了独立的 `trapframe` 和内核栈，回收时需单独释放。

2.6.3 `initproc` 对孤儿进程的回收

`wait()` 中，若当前进程是 `initproc`，则先调用 `reap_init_reparented_zombies(p)`。该函数扫描进程池，对所有 `parent == initproc` 且 `reparented_to_init` 标记为 1 的 ZOMBIE 子进程调用 `freeproc()`。这一机制确保所有被过继的僵尸进程最终都能得到回收。

2.6.4 线程退出与延迟回收

线程退出不能直接沿用进程级 `exit()`。当线程组中仍有其他线程时, `thread_exit()` 只结束当前线程, 保留共享的地址空间、文件表和其他线程的执行状态。尤其是当前线程仍运行在自己的内核栈上, 退出路径不能在调用现场直接释放该栈。

线程退出的关键路径如下:

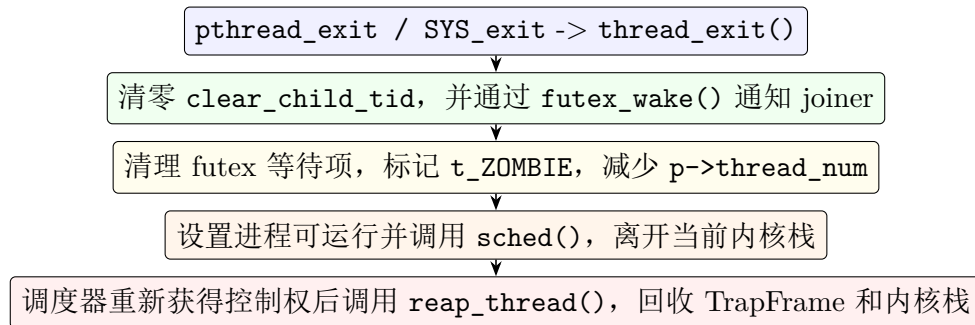


图 2.2: 线程退出与内核资源延迟回收路径

Algorithm 2.4: `thread_exit()` 与延迟回收算法

- 1: if 线程组中只有当前线程 then
- 2: 转入进程级 `exit()`, 结束整个进程
- 3: end if
- 4: if `clear_child_tid` 非空 then
- 5: 向用户地址写入 0, 并调用 `futex_wake()` 唤醒 joiner
- 6: end if
- 7: 调用 `futex_clear()` 清除当前线程的等待项
- 8: 设置 `t->state = t_ZOMBIE`, 并将 `p->thread_num` 减 1
- 9: 设置 `p->state = RUNNABLE`, 调用 `sched()` 切换到其他线程
- 10: 调度器返回后确认当前线程已离开执行现场
- 11: 调用 `reap_thread()` 释放线程私有 `TrapFrame` 和内核栈

`freeproc()` 仍负责整个进程进入 ZOMBIE 后的最终释放; `reap_thread()` 负责已经退出线程的独立资源。通过区分两条回收路径, 可以避免主线程 `TrapFrame` 重复释放, 也避免在线程仍使用自身内核栈时释放该栈。

2.7 本章小结

本章围绕进程资源、线程现场和 CPU 调度现场的分离, 完整说明了 `fork/clone` 创建、调度切换、`channel/futex` 阻塞、进程级与线程级信号, 以及进程退出和线程延迟回收路径。`proc_t`、`thread_t` 与 `cpu_t` 的分工使 `pthread` 同步、信号中断和资源回收能够在共享地址空间下协同工作。

第三章 内存管理

3.1 总体架构

内存子系统分为四层：Buddy 管理物理页，SLAB 管理内核对象，页表层维护映射，VMA 层描述虚拟区域语义。各层之间的关键约束是：VMA 记录的区域边界、页表中的实际映射和物理页的所有权必须同步变化；因此 mmap、munmap、缺页和进程复制不能只修改其中一层。

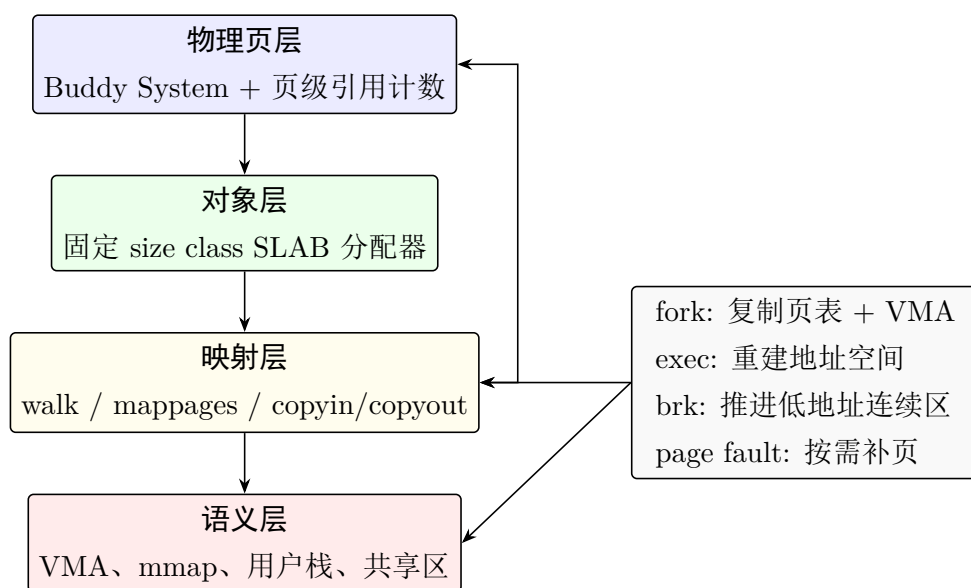


图 3.1: 内存子系统分层与主路径

初始化次序为 `pmem_init()` → `vmem_init()` → `slab_init()` (`kernel/sos_start_kernel.c`)。

3.2 Buddy System

3.2.1 数据结构

结构体	字段	类型	用途
<code>buddy_system_t</code>	—	—	全局 Buddy 系统实例，维护受管物理内存与各阶空闲链表。
<code>buddy_system_t</code>	<code>mem_start/end</code>	<code>uint64</code>	受管物理内存的起止地址。
<code>buddy_system_t</code>	<code>total_pages</code>	<code>uint64</code>	受管物理页总数。
<code>buddy_system_t</code>	<code>bitmap</code>	<code>uint64*</code>	页级位图，每位对应一页，其中 1 表示已使用，0 表示空闲。

buddy_system_t	nodes	buddy_node_t*	每页元数据数组,记录块起始地址、阶数和引用计数。
buddy_system_t	free_lists[11]	structlist	每个 order 对应的空闲块链表。
buddy_node_t	—	—	每页对应一项元数据,用于描述页所属块及空闲链表关系。
buddy_node_t	addr	uint64	所属块的起始物理地址。
buddy_node_t	order	int	块阶数,其中 -1 表示该页被元数据占用。
buddy_node_t	refcnt	int	引用计数,仅对 order 0 的物理页有效。
buddy_node_t	elem	list_elem	挂入对应空闲链表的链表节点。

表 3.2: Buddy System 核心数据结构

位图用于判断物理块是否仍被占用, `buddy_node_t` 用于恢复块的起始地址和阶数, 空闲链表则负责实际分配。释放和合并时必须同时验证这三类元数据, 避免已经摘链的节点再次被 `list_remove()`。

3.2.2 buddy_init() 自举过程

`buddy_init()` 将位图与节点数组元数据嵌入受管内存起始处, 随后按最高 order 切分剩余内存并挂入空闲链表。

3.2.3 buddy_alloc(): 分配算法

```

buddy_alloc(order):
    for current_order = order to MAX_ORDER:
        if free_lists[current_order] 非空:
            取出一个空闲块节点 (地址 = addr, 阶数 = current_order)

            // 分裂: 将过大的块逐级对半分割
            while current_order > order:
                current_order--
                block_size = PGSIZE << current_order
                buddy_addr = addr + block_size
                将 buddy_addr 作为新块插入 free_lists[current_order]
                set_buddy_free(buddy_addr, current_order)

            // 清空分配的内存
            memset(addr, 0, PGSIZE << order)
            set_block_metadata(addr, order)
            set_block_refcnt(addr, order, 1)
            set_buddy_used(addr, order)

```

```
    return addr
return NULL // 无可用内存
```

```
uint64 get_buddy_addr(uint64 addr, int order) {
    uint64 block_size = PGSIZE << order;
    return addr ^ block_size;
}
```

3.2.4 buddy_free(): 释放与合并算法

```
buddy_free(ptr, order):
    page_idx = (ptr - mem_start) / PGSIZE
    actual_addr = nodes[page_idx].addr // 从元数据恢复块起始
    actual_order = nodes[page_idx].order // 从元数据恢复块阶数

    // 重复释放检查
    if bitmap 中整个块已标记为空闲:
        return // 忽略重复释放

    set_buddy_free(actual_addr, actual_order)

    // 向上合并前验证伙伴的范围、位图、元数据和链表节点
    merge_addr = actual_addr
    merge_order = actual_order
    while merge_order < MAX_ORDER:
        buddy_addr = get_buddy_addr(merge_addr, merge_order)
        if !is_buddy_available(buddy_addr, merge_order):
            break
        buddy_node = nodes[buddy_addr]
        if buddy_node.addr != buddy_addr or buddy_node.order != merge_order:
            break
        if 伙伴节点不在 free_lists[merge_order] 中:
            若节点为空链状态则恢复, 否则终止本轮合并
            break
        从 free_lists[merge_order] 安全移除伙伴块
        清空伙伴节点的 prev/next
        merge_addr = min(merge_addr, buddy_addr)
        merge_order++
        set_block_metadata(merge_addr, merge_order)
    将最终合并块插入 free_lists[merge_order]
```

3.2.5 PAGE_NUM 的双架构差异

架构	PAGE_NUM	总内存	定义位置
RISC-V	250 * 1024	≈ 1000 MiB	include/hal/riscv/riscv.h
LoongArch	100 * 1024	≈ 400 MiB	include/hal/loongarch/loongarch.h

表 3.4: 双架构的 PAGE_NUM 配置

其中, RISC-V 的受管物理内存约为 1024 MiB, LoongArch 约为 400 MiB。LoongArch 的用户地址上限单独扩展到 64 GiB (`MAXUVA = 0x1000000000`), 以容纳 glibc、动态链接器和共享库的高地址映射; 这与页表本身的 48 位地址格式不是同一个概念。

3.2.6 页级引用计数

`pmem_inc_ref()` / `pmem_free_pages()` 通过 `refcnt` 管理共享页。引用计数仅对 `order 0` 有效, 大于 1 时只递减。

3.3 SLAB 分配器

3.3.1 固定 size class 设计

SLAB 分配器固定档位如下:

```
uint64 slab_size_table[FIXED_CACHE_LEVEL_NUM] = {
    8, 16, 32, 64, 128, 256, 512, 1024
};
```

每个 `kmem_cache` 维护 `free_slab` 和 `full_slab` 两条链表。`slab` 占用一页, 页头保存元数据。

档位 (bytes)	每 slab 对象数	利用率	典型用途
8	504	≈ 98%	极小结构体
16	252	≈ 98%	短指针数组
32	126	≈ 98%	链表节点
64	63	≈ 98%	共享内存元数据
128	31	≈ 97%	中等结构体
256	15	≈ 94%	文件系统元数据
512	7	≈ 87%	缓冲区
1024	3	≈ 75%	较大结构体

表 3.6: SLAB 固定档位的每页对象数

3.3.2 slab_alloc(): 分配流程

Algorithm 3.1: slab_alloc() 分配算法

```

1: 调用 __slab_size(size), 将请求大小对齐到最近固定档位
2: 通过 __fine_kmem_cache 查找对应的 kmem_cache
3: if 未找到对应 cache then
4:     panic                                ▷ 通常表示请求大小超过 1024
5: end if
6: if cache->free_slab 为空 then
7:     调用 __create_slab_with_size() 创建新 slab, 并挂入 free_slab
8: end if
9: 从 free_slab 中取出一个对象并更新空闲计数
10: if 该 slab 的空闲计数归零 then
11:     将该 slab 移入 full_slab
12: end if
13: 返回已分配对象

```

3.3.3 自举过程: simple_alloc → slab_init

simple_alloc() 是一个逐字节推进的 bump allocator:

```

simple_alloc(size):
    if simple_alloc_count + size > PAGE_SIZE:
        slab_boot_page = pmem_alloc_pages(1) // 新申请一页
        simple_alloc_count = 0
    alloc_ptr = slab_boot_page + simple_alloc_count
    memset(alloc_ptr, 0, size)
    simple_alloc_count += size
    return alloc_ptr

```

自举完成后设置 state = FULL, 后续分配均通过 slab_alloc().kmalloc()/kfree() 仍为按页分配。

3.4 页表与地址映射

3.4.1 walk(): 多级页表遍历核心算法

```

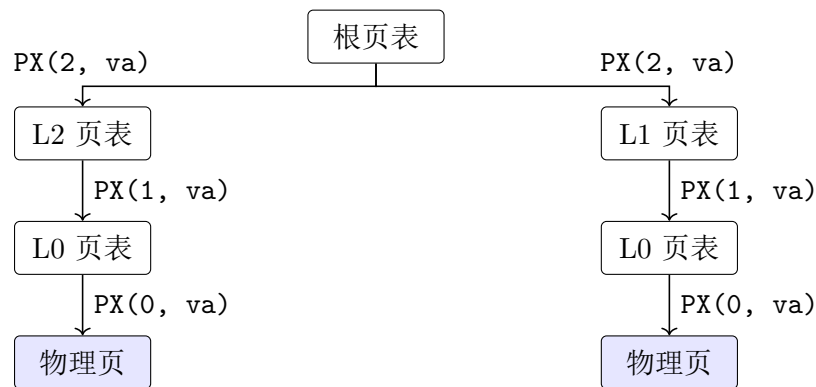
walk(pt, va, alloc):
    assert(va < MAXVA)
    for level = PT_LEVEL-1 down to 1:
        pte = &pt[PX(level, va)] // 取当前级索引
        if *pte & PTE_V: // 有效, 找下一级

```

```

    pt = (pgtbl_t)(PTE2PA(*pte) | dmwin_win0)
else if alloc: // 无效但允许分配
    pt = (pgtbl_t)pmem_alloc_pages(1)
    *pte = PA2PTE(pt) | PTE_WALK // 写入新页表物理地址
else:
    return NULL
pte = &pt[PX(0, va)] // 最后一级（叶节点）
return pte

```



RISC-V Sv39: 3 级 (9+9+9+12) **LoongArch:** 4 级 (9+9+9+9+12)

图 3.2: 多级页表遍历结构（图中为 RISC-V 三级示例）

3.4.2 RISC-V 与 LoongArch 的页表差异

维度	RISC-V (Sv39)	LoongArch
PT_LEVEL	3	4
虚拟地址宽度	39 位 (9+9+9+12)	48 位 (9+9+9+9+12), 用户映射上限为 64 GiB
MAXVA	1L << 38	1ULL << 46
物理地址访问	直接使用物理地址	通过 DMW 直映窗口 (dmwin_win0)
权限位模型	PTE_R/W/X 显式设置, 写页维护 PTE_D	PTE_NR/NX 逆逻辑 + PTE_W/PLV3, 软件维护 PTE_D
内核映射需求	需显式映射 UART/VirtIO/PLIC	DMW 窗口自动映射内核空间

表 3.8: 双架构页表差异对比

页表权限由 VMA 的 perm 转换而来，而不是在缺页时无条件授予读、写、执行权限。vm_protect() 修改已有叶级 PTE 时覆盖对应权限位；LoongArch 取消写权限时同时清

除 PTE_D，避免软件 TLB refill 把只读页面误判为可写。sys_mprotect() 将起始地址向下取整、结束地址向上取整，逐页更新权限并在完成后刷新 TLB。

3.5 VMA 与用户地址空间

3.5.1 地址空间布局

用户地址空间自低到高划分为：

常量	值 (RISC-V)	定义位置
	MAXUVA -	
USER_MMAP_START	0x10000000 - 2*PGSIZE	include/hal/riscv/riscv.h
USER_STACK_TOP	MAXUVA - PGSIZE	同上
USER_STACK_SIZE	100 * PGSIZE	include/kernel/vma.h
TRAMPOLINE	MAXVA - PGSIZE	include/hal/riscv/riscv_memlayout.h
TRAPFRAME	TRAMPOLINE - PGSIZE	同上

表 3.10: 用户地址空间关键常量

3.5.2 VMA 链表与区域语义

struct vma 保存区域类型、权限、起止地址、文件描述符与偏移。类型包括：

类型	用途	管理方式
NONE	哨兵头结点	永久存在于链表首位，标记链表边界
MMAP	mmap 映射区（文件映射/匿名映射）	alloc_mmap_vma() 创建，munmap() 裁剪
STACK	用户栈	alloc_vma_stack() 在 exec() 和 init_process() 中创建
SHARE	共享内存区域	通过共享内存系统调用创建

表 3.12: VMA 区域类型

vma_init() 创建双向循环链表，头结点 NONE，随后插入 MMAP 哨兵节点。VMA 同时记录页表权限、文件描述符和文件偏移，因此它既是地址区间索引，也是文件映射和缺页处理的语义来源。

3.5.3 alloc_vma / alloc_mmap_vma 的地址选择策略

```

mmap_choose_addr(p, len):
    lower = PGROUNDUP(p->sz)
    upper = USER_MMAP_START
    for vma 从链表尾部向前遍历:
        gap_low = max(vma->end, lower)
        if upper - gap_low >= PGROUNDUP(len):
            return PGROUNDDOWN(upper - PGROUNDUP(len))
        upper = vma->addr
    return MAP_FAILED

```

alloc_mmap_vma() 在所有 VMA 空隙中选择足够大的区间；使用 MAP_FIXED 时，则先由 vma_remove_range() 删除重叠区域和页表映射，再插入新的 VMA。这样动态链接器反复覆盖预留区时，VMA、PTE 和文件偏移仍保持一致。

3.5.4 mmap / munmap 的实现流程

munmap() 处理完整覆盖、前缀/后缀、中间段三种重叠。

Algorithm 3.2: munmap() 的 VMA 分裂算法

- 1: 在 VMA 链表中找到与区间 $[start, end)$ 重叠的映射项
- 2: **if** 解除范围完整覆盖原 VMA **then**
- 3: 删除该 VMA，并解除整段映射
- 4: **else if** 解除范围与原 VMA 前缀重叠 **then**
- 5: 将原 VMA 起点推进到 end
- 6: 同步调整文件偏移 f_off
- 7: **else if** 解除范围与原 VMA 后缀重叠 **then**
- 8: 将原 VMA 终点收缩到 $start$
- 9: **else**
- 10: 保留前段区间 $[addr, start)$
- 11: 新建后段区间 $[end, old_end)$
- 12: 设置后段的 $f_off += (end - start)$
- 13: **end if**
- 14: 对被解除的子区间执行页取消映射与物理页回收
- 15: 按页边界完成 TLB 刷新，避免处理器继续使用旧映射

3.5.5 VMA 并发保护与文件映射提交

VMA 元数据由进程的独立 vma_lock 保护，不复用负责进程生命周期和调度协议的 p->lock。所有修改链表或页表的公共入口先获取该锁，再调用带_locked 后缀的内部函数；内部函数不重复加锁，从而避免递归加锁。

```
sys_mmap / sys_munmap / page fault / fork / exec:
```

```
    acquire(&p->vma_lock)
```

```
    修改 VMA 区间、文件偏移或页表映射
```

```
    release(&p->vma_lock)
```

```
file-backed mmap:
```

```
    锁内建立 VMA_LOADING 区域
```

```
    锁外使用临时页执行 readat()
```

```
    再次加锁，确认 VMA 仍在链表中
```

```
    锁内建立 PTE 并复制临时页内容
```

```
    清除 VMA_LOADING
```

该边界避免持有 VMA 锁访问文件系统，同时防止并发的 munmap 或 exec 在文件读取期间使映射对象失效。/proc 地址空间遍历、退出释放和缺页处理也遵循同一把锁；VMA 锁只保护地址空间元数据，不替代未来多核场景下的跨 CPU TLB shutdown。

3.5.6 fork() 时 VMA 的共享与私有处理

复制进程时先在父进程的 VMA 锁保护下复制区域描述，再为子进程建立对应页表映射。共享映射增加物理页引用计数，私有映射复制物理页内容；VMA 的区域属性和页表权限随映射一并复制。

```
if vma_is_shared_mapping(vma): // MAP_SHARED 类型
```

```
    pmem_inc_ref(pa) // 增加页级引用计数
```

```
    mappages(new, start, pa, ...) // 同一物理页映射到子进程
```

```
else: // 私有映射
```

```
    mem = pmem_alloc_pages(1) // 分配新物理页
```

```
    memmove(mem, pa, PGSIZE) // 复制内容（写时复制尚未实现）
```

```
    mappages(new, start, mem, ...)
```

3.5.7 pagefault_handler 的决策流程

```
pagefault_handler(addr):
```

```
    acquire(&p->vma_lock)
```

```
    aligned_addr = PGROUNDDOWN(addr)
```

```
    pte = walk(p->pagetable, aligned_addr, 0)
```

```
    if pte is valid:
```

```
        if LoongArch and PTE_W is set:
```

```
            set PTE_D // PME 后补脏位并重试
```

```
            release(&p->vma_lock)
```

```
            return 0
```

```
    if PTE_W is clear:
        release(&p->vma_lock)
        return -1 // 真实的只读保护错误

    if addr <= p->sz:
        perm = low-area permission | PTE_U
    else if addr 落在某个 VMA:
        perm = vma->perm | PTE_U
    else:
        release(&p->vma_lock)
        return -1

    pa = pmem_alloc_pages(1)
    memset(pa, 0, PGSIZE)
    mappages(p->pagetable, aligned_addr, pa, PGSIZE, perm)
    release(&p->vma_lock)
    return 0
```

缺页路径只为目标区域建立允许的权限；LoongArch 的软件页表遍历可能在可写页的脏位未设置时触发 PME，此时补充 PTE_D 后恢复执行。VMA 权限、ELF 段权限和架构 PTE 规则共同决定最终访问属性。

3.6 本章小结

本章围绕内存管理主线，梳理了 Buddy、SLAB、页表与 VMA 四层机制。Buddy 提供页级物理内存分配与回收，并通过位图、节点元数据和链表一致性检查防止错误合并；SLAB 负责固定档位小对象管理；页表层承担虚实地址映射和架构权限转换；VMA 层补充区域边界、权限、用途与文件关联语义。

在用户地址空间路径中，mmap 能够在空闲区间选择地址并处理 MAP_FIXED 覆盖，munmap 能够维护 VMA 的分裂和文件偏移，缺页处理能够依据 VMA 权限建立页面或处理 LoongArch PME。独立的 vma_lock 将这些元数据操作串联起来，文件映射则通过临时页和锁内提交避免持锁访问文件系统。这样物理页所有权、页表映射和 VMA 语义在主要执行路径上保持一致。

第四章 硬件抽象层与异常处理机制

4.1 分层架构与代码边界

硬件抽象层(HAL)与硬件无关内核接口(HSAI)的分工,是本系统在双架构(RISC-V 64 与 LoongArch 64)条件下保持内核主干代码统一性的关键设计决策。

HAL 层承担以下三类底层事务:

1. 启动入口与特权级准备: `hal/riscv/entry.S`、`hal/riscv/start.c`、`hal/loongarch/entry.S`, 完成栈准备、CSR 配置后转入统一的 C 入口 `sos_start_kernel()`。
2. Trap 汇编桥接: 两套 `trampoline.S` (用户态陷入/返回)、`kernelvec.S` (内核态陷入/返回) 以及 LoongArch 的 `tlbrefill.S` (TLB 重填)。
3. 上下文切换原语: `swtch()`, 分别在 `hal/riscv/switch.S` 和 `hal/loongarch/swtch.S` 中实现, 仅保存 `ra`、`sp` 与被调用者保存寄存器。

HSAI 层将这些差异统一为少量接口函数, 主要位于 `hsai/hsai_trap.c` 与 `hsai/hsai_mem.c`。内核核心层(调度、系统调用、缺页、定时器)只调用 HSAI 函数, 不直接读写架构 CSR, 例如由 `hsai_get_arg()` 统一提取 `a0-a7`, 由 `hsai_usertrapret()` 统一准备用户态返回, 由 `hsai_config_pagetable()` 统一写入页表根寄存器。启动入口负责建立多 hart 初始化边界和每 CPU 启动栈, 但这只是进入统一内核路径的基础, 不等同于完整的 SMP 调度。`hsai_trap_init()` 进一步一次性配置中断入口和架构运行环境: RISC-V 开启 S 态外部中断、软件中断和内核 Trap 向量; LoongArch 设置 `ecfg`、`eentry`、`tlbrentry` 和机器错误入口, 并开启定时器中断。LoongArch 用户态运行 `glibc`、`jemalloc` 等代码时还需要启用 FPU、LSX 和 LASX, 否则 SIMD 指令会被误判为浮点异常。

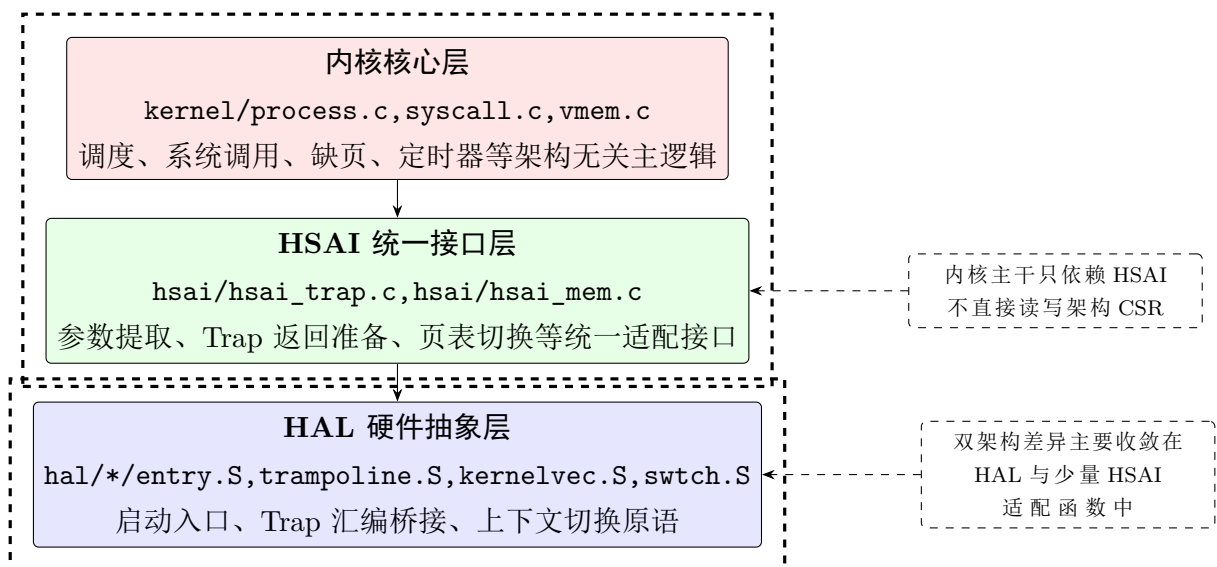


图 4.1: HAL → HSAI → Kernel 三层关系

4.2 双架构启动链

4.2.1 RISC-V 启动链

RISC-V 的启动入口在 `_entry` (`hal/riscv/entry.S`)，将 `sp` 指向 `entry_stack` 后调用 `start()`。

`start()` (`hal/riscv/start.c`) 在 M 态下设置 `mstatus.MPP` 为 S 态、配置 `mepc` 指向 `sos_start_kernel()`、清空 `satp`、通过 `medeleg` 和 `mideleg` 委托异常和中断给 S 态、配置 PMP，最后以 `mret` 跳转到 `sos_start_kernel()`。SBI 路径直接调用 `sos_start_kernel()`，由 QEMU 内置 OpenSBI 代为完成 M 态 CSR 配置。每个 hart 在 `entry.S` 中根据 hart id 选择独立启动栈；进入 C 入口后，首个启动 hart 负责清零 `.bss` 并完成基础初始化，其他 hart 等待 `riscv_kernel_ready` 后停在等待循环中，避免重复初始化或覆盖全局状态。

4.2.2 LoongArch 启动链

LoongArch 入口在 `hal/loongarch/entry.S` 的 `_entry`，进入 C 代码前在汇编层完成硬件配置：

1. 配置 `DMWIN0` (CA, PLV0, 地址窗口 0x9000 xxxx xxxx xxxx) 和 `DMWIN1` (UA, PLV0, 地址窗口 0x8000 xxxx xxxx xxxx)，建立直接映射窗口。
2. 配置 `CRMD` (PLV=0, IE=0, PG=1)，开启分页模式。
3. 清空 `TLBRENTY` 并执行 `invtlb` 使所有 TLB 项失效。
4. 读取 `CPUID` 构造每个 CPU 的启动栈。
5. 按 `CPUID` 选择独立启动栈，仅由 CPU0 清零 `.bss` 段。
6. 将当前 CPU id 保存在 `tp`，再跳转到 `sos_start_kernel()`。

LoongArch 入口只把 CPU0 清零 BSS、各 CPU 独立启动栈和 `tp` 传递作为早期初始化边界；后续共享资源初始化仍沿统一内核入口展开，本章不将其描述为完整的 SMP 初始化协议。

表 4.1: 双架构启动链对比

阶段	RISC-V (非 SBI)	LoongArch
入口文件	<code>hal/riscv/entry.S</code>	<code>hal/loongarch/entry.S</code>
M 态准备	<code>mstatus/mepc/medeleg/mideleg/PMP</code>	无 M 态，直启 S 态
地址映射	<code>satp=0</code> 关闭分页	<code>DMWIN0/1</code> 建立直接映射窗口
分页开启	在 <code>sos_start_kernel()</code> 内	入口配置 <code>CRMD.PG=1</code>
BSS 清零	boot hart 在 C 入口清零，其他 hart 等待 ready	CPU0 在入口汇编清零
启动栈与 CPU id	每 hart 独立栈，id 写入 <code>tp</code>	每 CPUID 独立栈，id 写入 <code>tp</code>

特权级转换	<code>mret (M → S)</code>	无需转换（直启 S 态）
进入内核	<code>sos_start_kernel()</code> 作为函数调用	同左

4.3 Trapframe 与 Context 的分工

`include/hsai/trap.h` 定义了 `struct trapframe`，用于保存完整的用户态执行现场。
`include/kernel/context.h` 定义了 `context_t`，仅保存调度切换所需的最小内核现场。

表 4.3: trapframe 域布局对比

域	RISC-V 偏移	LoongArch 偏移
<code>kernel_satp</code>	0	280
<code>kernel_sp</code>	8	248
<code>kernel_trap</code>	16	256
<code>epc/era</code>	24	264
<code>kernel_hartid</code>	32	272
<code>ra</code>	40	0
<code>sp</code>	48	16
<code>a0</code>	112	24
<code>a1-a7</code>	120-168	32-80
<code>s0-s11</code>	96-248	176-240
<code>t0-t6</code>	72-280	88-152

两类上下文分工如下：

- **trapframe**: 由 `uservec/userret` 在 `trampoline` 页中读写，保存用户态全部寄存器。进程从用户态陷入内核时，硬件跳转到 `uservec`，由汇编将现场保存至此；返回准备阶段再写入当前线程的 `kernel_sp`、内核 Trap 入口和 `kernel_hartid`。
- **context_t**: 由 `swtch()` 在 `hsai_swtch()` 中读写，仅保存 `ra`、`sp`、`s0-s11` (RISC-V) 或 `fp` (LoongArch)，不包含页表信息或用户态寄存器。

线程切换时，调度器先把线程私有的 `TrapFrame` 和 `context` 复制到进程当前活动镜像；`hsai_usertrapret()` 再依据当前线程内核栈设置 `kernel_sp`，并记录当前 `hart`。这样同一进程中的不同线程可共享地址空间，却不会误用其他线程的内核栈或返回现场。

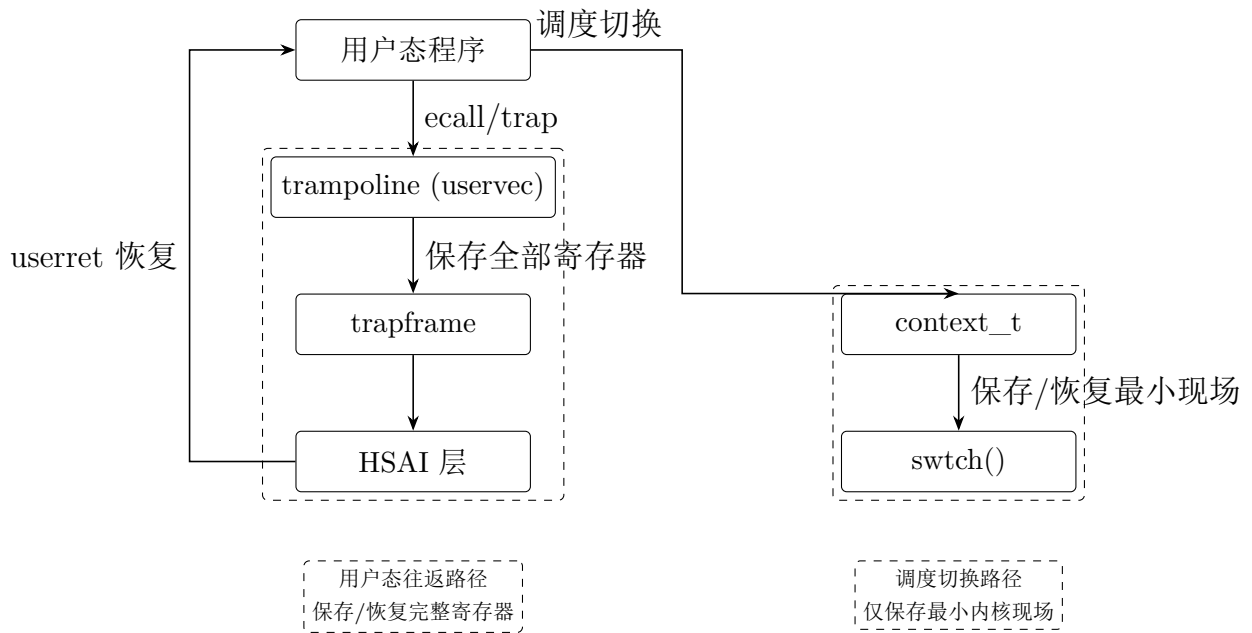


图 4.2: trapframe 与 context_t 的分工

4.4 用户态 Trap 主路径

用户态 Trap 主路径涵盖系统调用、缺页异常和设备中断三类事件。整体流程如算法 4.4 所示。

Algorithm 4.1: 用户态 Trap 主路径

- 1: 用户态执行过程中发生 `ecall`、异常或设备中断
- 2: 进入 `uservec()`，保存通用寄存器并切换到内核页表
- 3: 调用 `usertrap()`，保存异常 PC (`sepc` 或 `era`)
- 4: 调用 `devintr()` 检查是否为设备或时钟中断
- 5: **if** `devintr()` 返回非 0 **then**
- 6: **if** 中断类型为时钟中断 **then**
- 7: 更新用户态时间统计
- 8: 调用 `yield()` 让出 CPU
- 9: **end if**
- 10: 调用 `hsai_usertrapret()` 准备返回用户态
- 11: **else**
- 12: **if** 异常原因是系统调用 **then**
- 13: 将 `trapframe->epc/era` 前移到下一条指令
- 14: 开中断并调用 `syscall()` 分发系统调用
- 15: **else if** 异常原因是缺页异常 **then**
- 16: 调用 `pagefault_handler()` 尝试补页

```

17:      if 补页失败 then
18:          将进程标记为 SIGSEGV
19:      end if
20:  else
21:      打印异常信息并将进程标记为终止
22:  end if
23:      调用 hsai_usertrapret(), 写回返回所需寄存器与状态
24: end if
25: 跳转到 userret(), 执行 sret/ertn 返回用户态

```

4.4.1 usertrap() 异常分类

usertrap() (hsai/hsai_trap.c) 首先将内核 Trap 向量切换到 kernelvec (防止处理过程中再次陷入), 随后保存异常 PC (sepc 或 era), 然后根据异常原因分类。

表 4.5: RISC-V scause 与 LoongArch estat.ecode 对比

事件	RISC-V scause	LoongArch estat.ecode
系统调用	UserEnvCall (8)	0xb
Load 页故障	LoadPageFault (13)	0x1
Store 页故障	StorePageFault (15)	0x2
指令未对齐	InstructionMisaligned (0)	0x9
非法指令	IllegalInstruction (2)	0xa
断点	Breakpoint (3)	0xc
时钟中断	0x8000000000000005	TI_VEC 位
外部中断	(scause & 0xff) == 9	HWI_VEC 位

分类逻辑的伪代码如下 (RISC-V 分支):

```

usertrap(p->trapframe):
    stvec = kernelvec // 内核态陷入入口
    trapframe->epc = sepc // 保存异常PC

    which_dev = devintr()
    if which_dev != 0: // 设备/时钟中断
        if which_dev == 2: // 时钟中断
            p->utime++
            yield()
            hsai_usertrapret()
        // 其他中断后续处理

```

```

else: // 异常
    if cause == UserEnvCall: // 系统调用
        trapframe->epc += 4 // 跳过ecall
        intr_on()
        syscall(trapframe)
    elif cause == LoadPageFault
        or cause == StorePageFault:
        pagefault_handler(stval)
    else: // 不可恢复
        print_diagnostics()
        p->term_signal = SIGSEGV/SIGILL/SIGBUS
        exit(0)

```

LoongArch 分支结构相同，差异在于：异常 PC 字段为 era 而非 epc；系统调用判定使用 (estat&CSR_ESTAT_ECODE)>>16==0xb；页故障地址取自 badv 而非 stval；断点异常 (ecode 0xc) 先于系统调用判断，直接 era+=4 后跳过。

4.4.2 devintr() 中断分发

devintr() (hsai/hsai_trap.c) 将中断分为外部中断和时钟中断两类。

RISC-V 路径：检查 scause 高位置 1 且低 8 位为 9 (S 态外部中断) 则调用 plic_claim() 获取 IRQ 并路由至对应驱动；检查 scause==0x8000000000000005 (S 态时钟中断) 则调用 timer_tick()。

LoongArch 路径：读取 estat 和 ecfg，执行 estat&ecfg 位与得到有效中断向量。HWI_VEC 进入硬件中断分支，TI_VEC 进入定时器分支并调用 timer_tick()，随后写入 TICLR 清除中断标志。内核统一在 devintr() 中返回中断类别，上层 Trap 路径不依赖具体 CSR 名称。

4.4.3 平台定时器与诊断输出

定时器初始化需要适配平台实际提供的时间源。RISC-V virt 使用 Goldfish RTC 读取启动时间，并通过两次高位读取避免低位回绕；若时间值或频率不在合理范围，则使用默认启动时间和 DEFAULT_CLK_FREQ，避免访问不存在的设备导致启动失败。RISC-V 的下次中断写入 stimecmp 或通过 SBI 设置，LoongArch 则配置周期性倒计时定时器。

启动和设备诊断输出统一受 FINAL_DEV_DIAG 控制，正常评测路径不会把调试信息持续写入串口；需要定位启动、Trap 或 DMA 问题时再打开对应诊断开关。

4.4.4 缺页处理

pagefault_handler() (hsai/hsai_trap.c) 在 VMA 锁保护下判断故障地址是否在进程低地址连续区或某个 VMA 区间内：若需要补页，则按对应区域权限分配并清零物理页，再建立带 PTE_U 的映射；若发现已有叶级 PTE，则先区分 LoongArch PME 的脏位补全和真实的只读保护错误。分配失败或地址不在任何合法区域时返回 -1，由 usertrap()

将进程标记为 SIGSEGV。

4.5 内核态 Trap

`kerneltrap()` (`hsai/hsai_trap.c`) 处理内核态异常。与 `usertrap()` 的关键差异在于：

- 进入时检查当前模式是否为 S 态 (RISC-V 检查 `sstatus.SPP`, LoongArch 检查 `prmd.PPLV`)，否则 panic。
- 进入时要求中断已关闭 (`intr_get()==0`)。
- 不处理系统调用和用户态缺页，仅处理设备中断和时钟中断。
- 未识别的异常打印调试信息后 panic。
- 时钟中断递增 `p->ktime`，若进程状态为 `RUNNING` 则调用 `yield()`。
- 返回前恢复 `sepc/sstatus` (或 `era/prmd`)，因为 `yield()` 可能导致上下文切换。

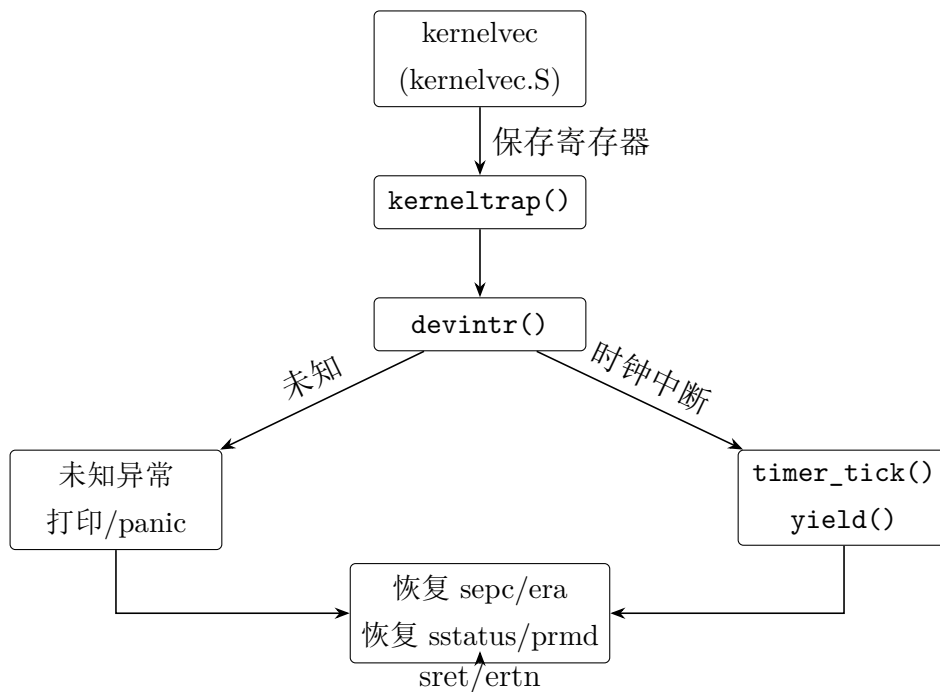


图 4.3: `kerneltrap()` 处理流程

4.6 LoongArch TLB Refill

LoongArch 缺乏 RISC-V 的硬件页表遍历能力，TLB 未命中时由软件在 `hal/loongarch/tlbrefill.S` 的 `handle_tlbr()` 中处理。该函数运行在专门的 TLB Refill Exception 入口 (`tlbrentry`)，不使用常规的 `eentry`。

```

handle_tlbr():
    // 保存 t0 到 TLBRSAVE
    csrwr t0, TLBRSAVE
  
```

```

// 读取 PGD (根页表地址)
csrrd t0, PGD

// 四级页表遍历
lddir t0, t0, 3 // 读 PGD[VPN3] -> PMD
srli.d t0, t0, 12
slli.d t0, t0, 12
lddir t0, t0, 2 // 读 PMD[VPN2] -> PUD
srli.d t0, t0, 12
slli.d t0, t0, 12
lddir t0, t0, 1 // 读 PUD[VPN1] -> PTE 基址
srli.d t0, t0, 12
slli.d t0, t0, 12

// 加载页表项
ldpte t0, 0 // 加载偶数页 PTE
ldpte t0, 1 // 加载奇数页 PTE

// 填充 TLB
tlbfill

// 恢复 t0 并返回
csrrd t0, TLBSAVE
ertn

```

LoongArch 的软件页表遍历由 `lddir` 读取下一级页表基址、`ldpte` 加载页表项、`tlbfill` 写入 TLB；相关路径还参与页面权限语义：当可写叶级 PTE 的 `PTE_D` 未设置时，存储访问可能触发 PME，`pagefault_handler()` 先补写 `PTE_D` 并重试，真正的只读访问则保持异常。RISC-V 与 LoongArch 的处理差异见表 4.7，LoongArch 还需在 `hsai_config_pagetable()` 中配置 `pwcl` 和 `pwch`。

表 4.7: TLB 未命中处理：RISC-V vs LoongArch

特性	RISC-V	LoongArch
页表遍历实现	硬件（页表遍历器）	软件（ <code>handle_tlbr</code> ）
TLB 缺失入口	无专用入口	<code>tlbrentry</code>
处理路径长度	无额外软件开销	约 $9 + 3N$ 条指令（ N = 页表级数）
页表级数	Sv39 (3 级)	4 级（使用 <code>lddir</code> 指令）

影响范围

无特殊设计需求

要求所有用户页表在内存中可达

4.7 hsai_usertrapret() 返回路径

`hsai_usertrapret()` (`hsai/hsai_trap.c`) 负责从内核态准备返回用户态，跳转到 `trampoline` 中的 `userret`。

Algorithm 4.2: `hsai_usertrapret()` 返回准备流程

- 1: 进入 `hsai_usertrapret()`
- 2: 关闭中断，避免返回准备过程中再次陷入
- 3: 重置用户态 Trap 入口，使后续用户态异常重新进入 `uservec()`
- 4: 根据当前线程选择内核栈，将栈顶写回 `trapframe->kernel_sp`
- 5: 将当前内核页表根写回 `trapframe` 中对应字段
- 6: 将内核 Trap 处理入口写回 `trapframe->kernel_trap`
- 7: 将当前 hart id 写回 `trapframe->kernel_hartid`
- 8: 设置返回用户态所需的 `sstatus/prmd`
- 9: 设置异常返回地址 `sepc/era`
- 10: 跳转到 `trampoline` 中的 `userret()`

准备完成后，通过函数指针跳转到 `trampoline` 内的 `userret`。参数传递和特权级返回指令的架构差异如下：

表 4.9: RISC-V `sret` vs LoongArch `ertn`

项目	RISC-V	LoongArch
返回地址寄存器	<code>sepc</code>	<code>era</code>
页表寄存器	<code>satp</code> (写入 <code>a1</code>)	<code>pgdl</code> (写入 <code>a1</code>)
Trapframe 寄存器	<code>a0 = TRAPFRAME</code>	<code>a0 = TRAPFRAME</code>
返回指令	<code>sret</code>	<code>ertn</code>
状态恢复	<code>sstatus</code> (由 <code>sret</code> 自动恢复)	<code>prmd</code> (由 <code>ertn</code> 自动恢复)
Trampoline 段	<code>hal/riscv/trampoline.S</code>	<code>hal/loongarch/trampoline.S</code>

`userret` 在汇编层完成用户页表切换、用户通用寄存器恢复，执行 `sret` 或 `ertn` 恢复 CPU 状态并跳转到 `sepc/era` 处继续执行。

4.8 本章小结

本章通过 HAL、HSAI 与内核主干的分层，统一了 RISC-V 和 LoongArch 的启动、Trap、中断、页表和用户态返回路径。多 hart 启动同步、TrapFrame 与 context 分工、用户态缺页、LoongArch TLB Refill/PME 以及 `userret` 的特权级返回共同构成从硬件入口到用户程序恢复执行的完整链路；架构差异被限制在底层入口和 HSAI 接口内，内核核心路径保持统一。

第五章 系统调用与用户态接口

5.1 设计目标

系统调用层是用户态与内核之间的 ABI 边界。系统调用设计遵循以下两条原则：

1. 以 **Linux 风格系统调用号**为起点。include/kernel/syscall_ids.h 中的编号直接对应 Linux 的 syscall table，按 BusyBox、glibc、Iozone、LTP、pthread 等工作负载逐步补齐。内核无需源码级修改用户程序，即可运行预编译的 Linux ELF 二进制。
2. **双架构差异隔离在系统调用边界**。陷入指令和寄存器布局由调用桩与 HSAI 处理，参数顺序和用户结构布局的差异在 kernel/syscall.c 入口处归一化；兼容路径和用户空间搬运集中在 kernel/syscall.c 与 kernel/vmem.c 中。

系统调用与用户态接口按以下分层组织：

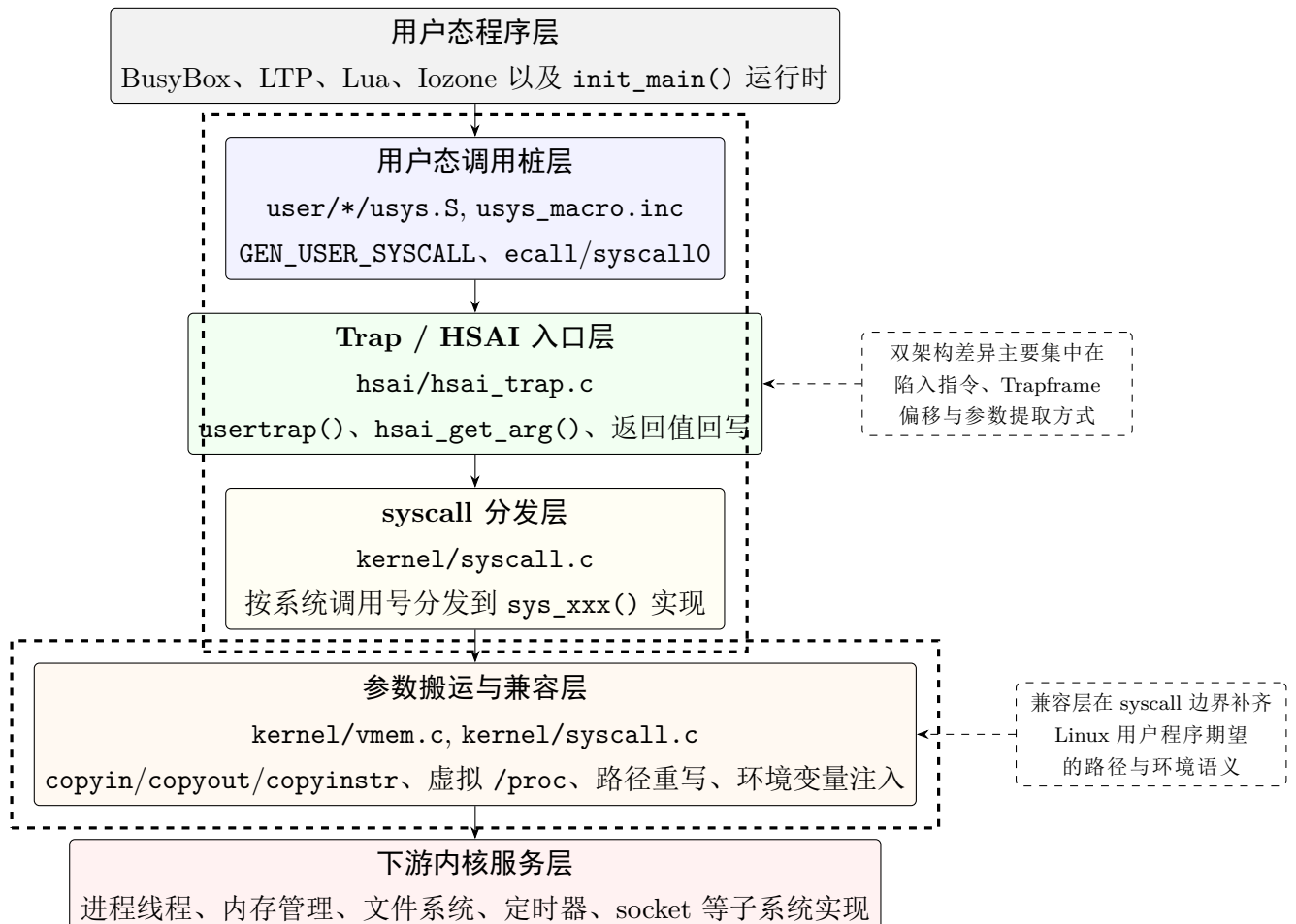


图 5.1: 系统调用与用户态接口的分层结构

5.2 ABI 入口与用户态调用桩

5.2.1 双架构调用约定

用户态调用桩由宏 `GEN_USER_SYSCALL` 生成。RISC-V 版本位于 `user/riscv/include/usys_macro.inc`, LoongArch 版本位于 `user/loongarch/include/usys_macro.inc`。两者遵循同一抽象语义：将系统调用号写入约定寄存器，执行陷入指令，返回调用者。

表 5.1: 当前系统调用 ABI 的双架构约定

含义	RISC-V 64	LoongArch 64
系统调用号	a7	\protect\TU\textdollar a7
参数寄存器	a0–a5	\protect\TU\textdollar a0– \protect\TU\textdollar a5
返回值寄存器	a0	\protect\TU\textdollar a0
陷入指令	ecall	syscall0 jirl\protect\TU\textdollar ze
返回指令	ret	ro,\protect\TU\textdollar ra, 0

`user/riscv/usys.S` 与 `user/loongarch/usys.S` 覆盖内核自带用户程序所需的最小接口集合，例如 `sys_execve`、`sys_openat`、`sys_read`、`sys_mmap` 等。

5.2.2 GEN_USER_SYSCALL 宏

调用桩生成宏（RISC-V 版本）如下：

```
.macro GEN_USER_SYSCALL name, num
.globl \name
\name:
    li a7, \num // 将系统调用号装入 a7
    ecall // 陷入内核
    ret // 返回用户态调用者
.endm
```

LoongArch 版本仅将 `ecall` 替换为 `syscall0`，确保所有用户态系统调用使用统一的陷入点，且不修改 `a0–a5` 参数寄存器的值。

5.3 从 Trap 到 syscall() 的统一分发

5.3.1 syscall() 分发器

`syscall()` (`kernel/syscall.c`) 调用 `hsai_get_arg()` 提取 `a[0..7]`，以 `a[7]` 为编号经 `switch` 分发到 `sys_xxx()`，返回值写入 `trapframe->a0`。

```
void syscall(struct trapframe *trapframe) {
```

```
for (int i = 0; i < 8; i++)
    a[i] = hsai_get_arg(trapframe, i);

long long ret = -1;
switch (a[7]) {
case SYS_write: ret = sys_write(a[0], a[1], a[2]); break;
case SYS_read: ret = sys_read(a[0], a[1], a[2]); break;
case SYS_openat: ret = sys_openat(a[0], (char*)a[1], ...); break;
case SYS_mmap: ret = sys_mmap(a[0], a[1], a[2], ...); break;
// ... (约 100+ 个 case)
default: ret = -ENOSYS; break;
}
trapframe->a0 = ret;
}
```

hsai_get_arg() 统一参数抽象的实现如下:

```
uint64 hsai_get_arg(struct trapframe *trapframe, uint64 num) {
    switch (num) {
    case 0: return trapframe->a0;
    case 1: return trapframe->a1;
    // ... a2..a6
    case 7: return trapframe->a7;
    }
}
```

RISC-V 与 LoongArch 的 trapframe 中 a0-a7 偏移不同, HSAI 函数屏蔽这差异。

5.4 参数搬运机制

指针参数需要借助 `kernel/vmem.c` 提供的用户空间数据搬运函数。

表 5.3: 参数搬运函数职责分工

函数	功能	典型用途
<code>copyin()</code>	从用户空间复制任意长度数据到内核	<code>sys_writev</code> 复制 <code>iovec</code>
<code>copyout()</code>	从内核复制数据到用户空间	<code>sys_gettimeofday</code> 、 <code>sys_statx</code>
<code>copyinstr()</code>	从用户空间复制以 <code>\0</code> 结尾的字符串	<code>sys_openat</code> 复制路径名
<code>fetchaddr()</code>	从用户地址读取一个指针宽度整数	<code>sys_execve</code> 读取 <code>argv[i]</code>
<code>fetchstr()</code>	从用户空间读取字符串（基于 <code>copyinstr()</code> ）	<code>sys_execve</code> 读取参数值

跨页搬运基于 `walkaddr()` (`kernel/vmem.c`) 实现：将目标用户地址向下页对齐后逐页翻译得到物理地址，再逐页执行 `memmove()`。搬运函数不感知架构差异，`walkaddr()` 内部已处理了双架构页表翻译路径的差异。

5.5 Linux 用户态 ABI 兼容边界

系统调用号能够被分发，只是 ABI 兼容的起点。用户态还会检查参数顺序、结构体大小、字段布局、输入输出方向和错误返回。内核因此在系统调用边界显式区分用户态结构和内核内部结构：先用 `copyin()` 将用户数据搬入临时内核对象，完成语义检查和内部转换，再用 `copyout()` 按 Linux 用户结构实际大小返回结果。`sys_fstatat`、`sys_statx`、`sys_statfs` 等文件状态接口，以及 `sys_rt_sigaction`、`sys_rt_sigprocmask` 等信号接口，都沿用这一方向；字符串参数始终先经过 `copyinstr()`，避免直接解引用用户指针。

线程运行时还依赖一组不能只用“占位返回值”处理的接口。`sys_gettid()` 返回线程 ID，`sys_set_tid_address()` 建立线程退出时清理地址的关联，`sys_sigaltstack()` 使用显式的 `ss_sp/ss_flags/ss_size` 布局，`sys_riscv_hwprobe()` 则按用户数组逐项读回并写回探测结果。这些接口规模不大，却共同决定了 `glibc`、`musl` 和 `pthread` 运行时能否完成启动、线程创建与异常路径初始化。

用户态 wrapper

- > 架构陷入入口与 `TrapFrame` 取参
- > `syscall()` 按系统调用号分发
- > `copyin/copyinstr` 与 ABI 结构转换

-> `sys_xxx()` 调用进程、内存或文件服务
 -> 按用户结构大小 `copyout`, 回写 `a0`

5.5.1 clone 参数归一化

`clone` 是架构 ABI 差异必须停留在边界处的典型例子。RISC-V 的用户态约定将参数解释为 `(flags, stack, ptid, tls, ctid)`, LoongArch 则将后两个位置解释为 `(flags, stack, ptid, ctid, tls)`。`sys_clone()` 在进入公共线程创建逻辑前先得到统一的 `clone_tls` 和 `clone_ctid`, 随后根据 `CLONE_VM` 选择 `clone_thread()` 或进程创建路径。这样 TLS、`pthread_self()` 和 `clear_child_tid` 的语义由同一套线程实现维护, 不会因为参数错位在更晚的 `join` 或线程退出阶段才暴露。

```
sys_clone(flags, stack, ptid, tls, ctid):
    if stack == 0:
        return fork()
#ifdef RISC_V
    clone_tls = tls
    clone_ctid = ctid
#else
    clone_tls = ctid
    clone_ctid = tls
#endif
    if flags & CLONE_VM:
        return clone_thread(stack, ptid, clone_tls, clone_ctid, flags)
    return clone(flags, stack, ptid, clone_tls, clone_ctid)
```

5.5.2 TCGETS: 终端结构的定长兼容

终端接口体现了结构体 ABI 中“长度也属于语义”的约束。glibc 的用户态终端结构包含 32 个控制字符槽位, 而 Linux 内核 TCGETS ABI 使用 19 个槽位。若内核按照用户态结构的大小直接写回, 额外数据会覆盖 glibc 调用栈上的保护值, 最终表现为用户态的 stack smashing, 而不是一个显眼的系统调用错误。当前实现定义专用的 `linux_termios_s_local`, 按四个 32 位字段、一个行规程字节和 19 个控制字符组成 36 字节缓冲区, 并通过 `ioctl_copyout_value()` 只复制 `sizeof(tio)` 字节。

表 5.5: TCGETS 的结构布局与输出边界

层次	结构或布局	输出长度	内核处理
Linux 内核 ABI	四个 <code>uint32</code> 、 <code>c_line</code> 、 <code>c_cc[19]</code>	36 字节	<code>linux_termios_local</code> 清零后返回

glibc 用户封装	用户结构保留 c_cc[32]	由 wrapper 转换	接收内核 36 字节结果后再完成 用户态布局转换
系统调用边界	TCGETS 的 argp 指针	sizeof(tio)	copyout() 失败返回 -EFAULT, 不扩大写回范围

5.6 系统调用分类

当前系统调用按编号分布在 include/kernel/syscall_ids.h 中，实现集中于 kernel/syscall.c。从主要工作负载来看，它们按以下子系统组织：

表 5.7: 系统调用的主要类别

类别	代表接口	主要依赖
进程与执行	sys_fork、sys_clone、sys_wait、 sys_execve、sys_exit、sys_kill、 sys_tgkill、sys_sched_yield sys_brk、sys_mmap、sys_munmap、 sys_mprotect、	进程/线程管理、信号、ELF 装 载器
内存与时间	sys_clock_gettime、 sys_clock_nanosleep、 sys_gettimeofday、sys_settimer sys_openat、sys_read、 sys_write、sys_readv、 sys_pread、sys_getdents64、	页表、VMA、定时器、调度
文件与路径	sys_fstat、sys_statx、 sys_readlinkat、sys_mkdirat、 sys_unlinkat、sys_mount、 sys_umount、sys_renameat2 sys_rt_sigaction、 sys_rt_sigprocmask、sys_futex、 sys_ppoll、 sys_pselect6_time32、	VFS/EXT4、路径解析、文件 对象层
信号与同步	sys_eventfd2、 sys_epoll_create1、 sys_signalfd4、 sys_timerfd_create、 sys_pidfd_open	信号处理、futex、等待队列

	sys_socket、sys_bind、	
	sys_connect、sys_accept、	
网络与系统信息	sys_sendto、sys_recvfrom、	socket 层、系统信息填充
	sys_sysinfo、sys_uname、	
	sys_getrandom、sys_prlimit64	

5.7 兼容层设计

兼容层使预编译的 Linux ELF 程序无需修改即可运行，在系统调用边界上补齐路径、元数据、环境变量和挂载语义。

5.7.1 用户态运行时视图与虚拟文件接口

BusyBox、glibc 和诊断工具依赖读取 `/proc` 和 `/etc` 下的运行时视图。内核通过 `is_busybox_virtual_path()` (`kernel/syscall.c`) 识别这些路径，在 `sys_openat()`、`sys_fstatat()`、`sys_statx()` 等函数中创建 `FD_BUSYBOX` 文件对象或返回合成元数据；内容由当前进程、地址空间和文件状态生成，不经过底层 EXT4 的普通文件读取路径。

当前支持的虚拟路径包括：

```
/proc /proc/self /proc/mounts
/proc/meminfo /proc/cpuinfo /proc/config.gz
/proc/self/maps /proc/self/exe /proc/self/status
/proc/self/stat /proc/[pid]/stat /proc/sys/kernel/pid_max
/etc/passwd /etc/group /dev/misc/rtc
/proc/sys/fs/pipe-user-pages-soft /proc/sys/kernel
```

处理流程如算法 5.7.1 所示。

Algorithm 5.1: BusyBox 虚拟文件处理流程

- 1: 进入 `sys_openat()` 或相关路径查询接口
- 2: 读取并规范化目标路径
- 3: **if** `is_busybox_virtual_path()` 判定该路径为虚拟路径 **then**
- 4: 构造合成文件对象或合成元数据
- 5: 将文件类型标记为 `FD_BUSYBOX`
- 6: 直接返回虚拟文件描述符或查询结果
- 7: **else**
- 8: 将请求下发到底层文件系统
- 9: 调用 `vfs_ext4_openat()` 或对应 EXT4 路径处理函数
- 10: 返回真实文件描述符或查询结果
- 11: **end if**

以 `fill_busybox_virtual_kstat()` (`kernel/syscall.c`) 为例，它在 `sys_fstatat()`

中填充 `kstat` 结构的 `st_mode`、`st_size` 等字段。例如 `/proc` 构造为权限 `0040777` 的目录，`/proc/meminfo` 构造为权限 `0100644` 的普通文件。

5.7.2 动态 maps 与真实执行路径

`/proc/self/maps` 不能用一份固定文本代替，否则动态链接器无法判断当前 ELF、共享库、栈和匿名映射的边界。`kernel/fs/file.c` 中的 `busybox_virtual_proc_self_maps()` 先记录进程的程序映射，再在 `vma_lock` 保护下遍历 VMA 链表，将 `PTE_R/PTE_W/PTE_X` 转换为 Linux 风格的权限字符串，最后由 `busybox_virtual_read()` 按文件偏移通过 `copyout()` 返回用户缓冲区。

```
read("/proc/self/maps")
-> 识别 FD_BUSYBOX 虚拟文件
-> 加入当前 ELF 映射和 exe_path
-> 持 vma_lock 遍历 VMA，格式化区间与 rwxp 权限
-> 按 f_pos 截取内容并 copyout 到用户缓冲区
```

`/proc/self/exe` 也不再依赖固定路径：`vfs_ext4_readlink()` 从当前进程的 `exe_path` 返回真实执行路径，供动态链接器解析程序身份和相对加载关系。类似地，`sys_prlimit64()` 为栈、地址空间、文件描述符等资源提供合理的默认限制，并允许运行时查询或更新可修改的限制；这些小接口共同构成用户态启动环境的一部分。

5.7.3 LD_LIBRARY_PATH 注入

`sys_execve()` (`kernel/syscall.c`) 在复制 `argv` 与 `envp` 时，预先在环境变量头部注入 `LD_LIBRARY_PATH=/glibc/lib:/musl/lib:`，并跳过用户态已提供的同名环境变量。

```
sys_execve(path, uargv, uenvp):
    // 复制参数数组到 argv[]
    for i = 0..MAXARG:
        fetchaddr(uargv + i*8, &uarg)
        if uarg == 0: break
        fetchstr(uarg, argv[i], PGSIZE)

    // 注入 LD_LIBRARY_PATH
    envp[0] = "LD_LIBRARY_PATH=/glibc/lib:/musl/lib:"
    env_count = 1

    // 复制用户环境变量（跳过同名项）
    for i = 0..:
        fetchaddr(uenvp + i*8, &uenv)
        if uenv == 0: break
        if strncmp(env_str, "LD_LIBRARY_PATH=", 16) == 0:
```

```

        continue // 跳过
    envp[env_count++] = env_str

    ret = exec(path, argv, envp)
    // 清理分配的内存

```

这一机制使后续装载器可以在相同 ABI 边界下同时服务 glibc 与 musl 风格程序，而不要求每个用户程序在启动前都手工准备共享库搜索路径。

5.7.4 /dev/shm 重映射

共享内存工作负载（如 LTP 的 shmctl 测例）期望 /dev/shm 存在于 tmpfs 上。get_absolute_path()（kernel/fs/inode.c）在路径规范化时将 /dev/shm 及其子路径重写到 /ltp-shm，统一重定向到根文件系统下。

5.7.5 Synthetic Mount 机制

sys_mount() 和 sys_umount()（kernel/syscall.c）支持 tmpfs、ext4 和 vfat 三种类型。对于非根目录的挂载请求，内核不注册第二个文件系统实例，而是通过 set_synthetic_ro_mount() 维护 synthetic mount 状态。

```

sys_mount(special, dir, fstype, flags, data):
    copyinstr 路径到内核缓冲区
    get_absolute_path(dir) -> abs_path

    if fstype == "tmpfs":
        set_synthetic_ro_mount(abs_path)
        synthetic_mount_readonly = (flags & MS_RDONLY)
        return 0

    if fstype in ("ext4", "vfat") and abs_path != "/":
        set_synthetic_ro_mount(abs_path) // 非根目录：模拟挂载
        synthetic_mount_readonly = (flags & MS_RDONLY)
        return 0

    // 根目录：真实挂载
    return fs_mount(TMPDEV, fstype, abs_path, flags, data)

```

后续对该路径的写入请求会被 reject_path_write_on_ro_mount() 拦截，并返回 -EROFS。sys_umount() 检测到 synthetic mount 路径时，直接清除相应挂载状态，而不操作底层 VFS。

5.8 本章小结

当前系统调用层形成了完整的调用路径：用户态调用桩触发陷入，Trap 层完成异常识别，`syscall()` 分发到 `sys_xxx()`，借助 `copyin/copyout/copyinstr` 搬运数据，并在结构体和参数边界完成 Linux ABI 转换，最后恢复到用户态。`clone` 的架构参数归一化和 TCGETS 的定长写回说明了兼容层如何处理容易被忽略的布局约束；动态 `/proc/self/maps`、真实 `/proc/self/exe`、路径重写、环境变量注入和 `synthetic mount` 则为动态链接器、BusyBox 与系统诊断工具提供运行时视图。

第六章 文件系统与程序装载

6.1 总体架构

文件系统与程序装载的实现分布在 `kernel/fs/` 与 `kernel/exec.c` 中，按分层方式组织：

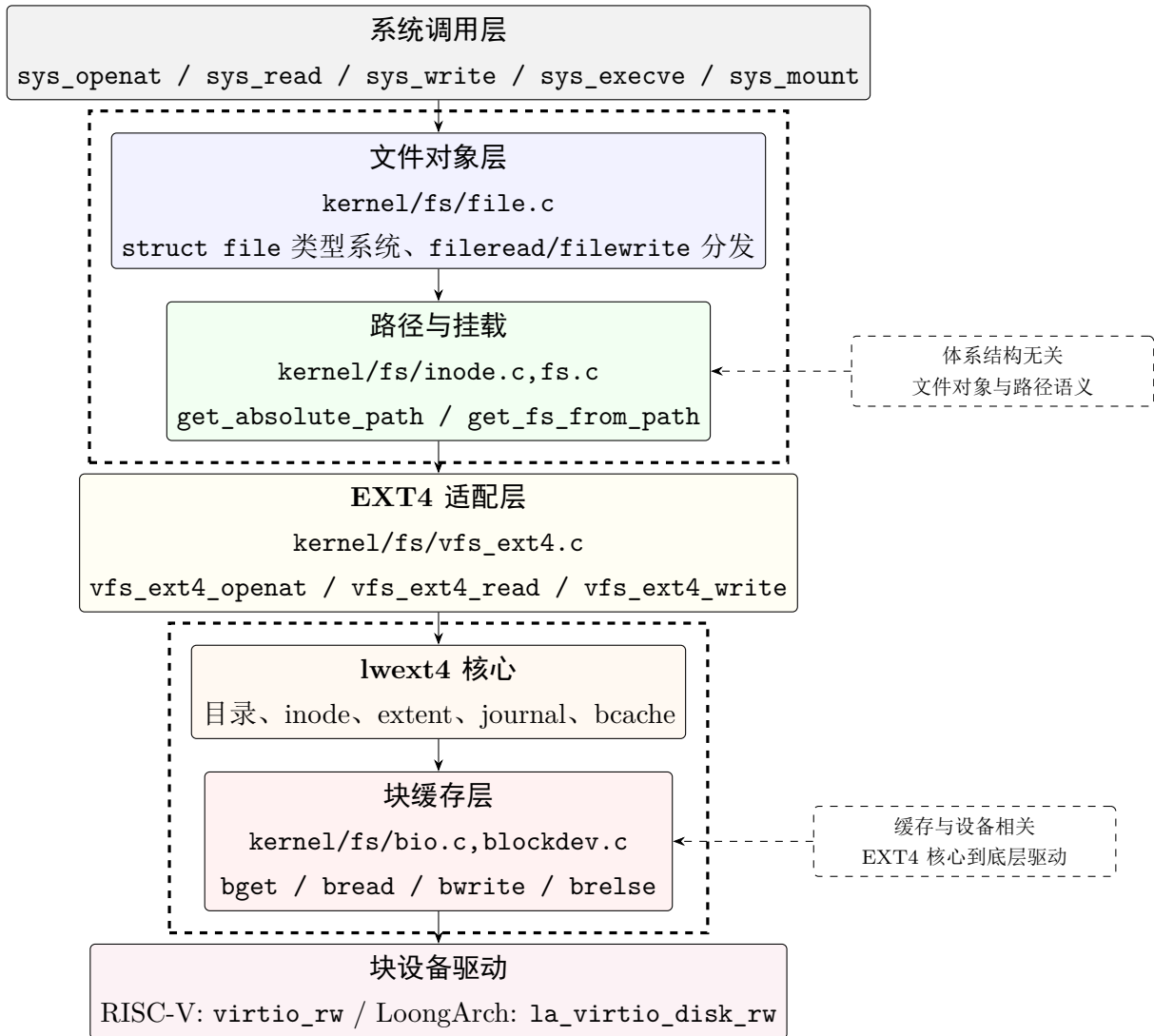


图 6.1: 文件系统访问栈的分层结构

各层职责如下：

- **文件对象层** (`kernel/fs/file.c`)：提供统一的 `struct file` 抽象，管理文件描述符的分配、复制、关闭和类型分发 (`FD_PIPE` / `FD_REG` / `FD_DEVICE` / `FD_BUSYBOX` / `FD_SOCKET` 等)。
- **路径与挂载** (`kernel/fs/inode.c, fs.c`)：将用户传入的路径归一化为绝对路径，通过最长前缀匹配选择挂载点。

- **EXT4 适配层** (`kernel/fs/vfs_ext4.c`): 将文件对象层的语义请求转换为 `lwext4` 库调用。
- **lwext4 核心**: 完成 EXT4 文件系统格式解析、inode 操作、目录遍历和日志 (journal) 管理。
- **块缓存层** (`kernel/fs/bio.c`): 以 LRU 链表维护磁盘块的缓存副本, 对上层提供 `bread / bwrite` 接口, 屏蔽底层设备差异。
- **块设备驱动**: RISC-V 通过 MMIO 驱动 VirtIO 设备, LoongArch 通过 PCI/virtio-pci 方式驱动。

6.2 文件对象层

6.2.1 struct file 的类型系统

`include/kernel/fs/file.h` 中的 `struct file` 是文件访问的统一抽象。其 `f_type` 字段标识对象类型:

类型常量	数值	用途
<code>FD_NONE</code>	0	未分配
<code>FD_PIPE</code>	1	管道
<code>FD_REG</code>	2	普通文件
<code>FD_DEVICE</code>	3	字符设备
<code>FD_BUSYBOX</code>	4	BusyBox 虚拟文件
<code>FD_SOCKET</code>	5	Socket (stub)
<code>FD_EPOLL</code>	6	epoll (stub)
<code>FD_EVENTFD</code>	7	eventfd
<code>FD_SIGNALFD</code>	8	signalfd (stub)
<code>FD_TIMERFD</code>	9	timerfd (stub)
<code>FD_PIDFD</code>	10	pidfd (stub)

表 6.2: 文件对象类型枚举

`struct file` 通过联合体 `f_data` 将不同后端指向同一入口:

```
struct file {
    enum { FD_NONE, FD_PIPE, FD_REG, FD_DEVICE, FD_BUSYBOX, ... } f_type;
    int f_flags; // O_RDONLY / O_WRONLY / O_RDWR
    int f_count; // 引用计数
    uint f_pos; // 读写偏移
    int f_major; // 设备号 (FD_DEVICE 专用)
    int removed; // 延迟删除标记
    char f_path[MAXPATH];
    union {
```

```
    struct pipe *f_pipe; // FD_PIPE
    file_vnode_t f_vnode; // FD_REG / FD_DEVICE
} f_data;
};
```

6.2.2 生命周期管理

函数	语义
<code>filealloc()</code>	从全局表 <code>f_table[]</code> 分配引用计数为 1 的空文件对象
<code>fdalloc(f)</code>	在进程 <code>ofile[]</code> 中找空槽，将文件对象绑定到文件描述符
<code>filedup(f)</code>	增加文件对象引用计数 (<code>f_count++</code>)，用于 <code>dup/fork</code>
<code>fileclose(f)</code>	减少引用计数：归零时调用后端关闭逻辑

`fileclose` 的关键行为是延迟删除：先持有 `f_table.lock` 递减 `f_count`；归零后将状态保存到栈上，清空原对象，再根据 `f_type` 分发到对应后端的关闭逻辑。若 `removed` 标记为真，额外调用删除操作。

6.2.3 读写类型分发

`fileread` 和 `filewrite` 按 `f_type` 分发：

普通文件与目录请求转入 EXT4 适配层，设备文件转入字符设备接口，管道转入管道子系统，BusyBox 虚拟文件则直接由内核生成内容。借助这一统一入口，系统调用层无需感知后端差异即可完成读写分发。

6.2.4 FILE_OPS 分发表

系统通过全局结构体 `FILE_OPS` 将文件操作集中管理：

```
struct file_operations FILE_OPS = {
    .dup = &filedup,
    .close = &fileclose,
    .read = &fileread,
    .readat = &filereadat,
    .write = &filewrite,
    .fstat = &filestat,
    .statx = &filestatx,
    .writable = &filewriteable,
    .readable = &filereadable,
};
```

6.3 EXT4 适配层

`kernel/fs/vfs_ext4.c` 是将内核文件对象与 `lwext4` 库连接的关键适配层。它不重新实现 EXT4 语义，而是将内核侧的数据结构转换为 `lwext4` API 的参数。

6.3.1 打开路径：`vfs__ext4__openat`

`vfs_ext4_openat` 的处理流程：

1. 调用 `vfs_ext4_is_dir` 判断目标路径是否为目录（通过 `ext4_raw_inode_fill` 读取原始 inode，检查 `EXT4_INODE_MODE_DIRECTORY`）；
2. 若是目录：分配目录 vnode（`vfs_alloc_dir`），调用 `ext4_dir_open`；
3. 若是文件：分配文件 vnode（`vfs_alloc_file`），调用 `ext4_fopen2`；
4. 再次读取原始 inode，若发现字符设备类型，则将文件类型改为 `FD_DEVICE` 并记录 `f_major`；否则保持 `FD_REG`。

6.3.2 读写路径：分页中转缓冲区

`vfs_ext4_read` 和 `vfs_ext4_write` 采用分页式中转机制处理用户空间地址：

```
int vfs_ext4_read(struct file *f, int user_addr,
                  const uint64 addr, int n)
{
    char *buf = kalloc(); // 分配一页作为中转缓冲区
    for (has_read = 0; has_read < n;
         has_read += realread, uaddr += realread)
    {
        expect = min(n - has_read, PGSIZE);
        ext4_fread(ext4_f, buf, expect, &realread); // 读入内核页
        either_copyout(user_addr, uaddr, buf, realread); // 拷贝到用户空间
    }
    kfree(buf);
    f->f_pos = ext4_f->fpos;
    return byteread;
}
```

设计理由：`lwext4` 的 `ext4_fread` 接受连续内核缓冲区作为目标，而用户地址空间可能不连续。因此需要以 `PGSIZE` 为粒度，通过内核页作为中介进行逐段拷贝。

6.3.3 EXT4 大文件与稀疏文件读取

文件读取不能把逻辑块号直接当作磁盘上的连续偏移。`ext4_fread()` 先从 inode 和 extent 映射中查询逻辑块对应的物理块，再将连续的有效物理块合并后交给块设备读取；当映射结果为物理块号 0 时，将其视为空洞并向输出缓冲区填零，不能继续读取块设备上的固定位置。对于未对齐的首块和不足一个块的文件尾部，读取路径按实际剩余长度处理，分页装载和中转缓冲区中未被文件内容覆盖的部分保持为零。

这一处理同时覆盖大文件的块范围读取、稀疏文件和 ELF 文件尾部的 BSS 场景：有效数据经过 `ext4_block_readbytes()` 或连续块读取进入缓冲区，空洞直接清零，文件尾部不会把超级块或其他块内容误当作用户数据。因而动态链接器读取共享库、重定位表或程序尾部时，底层 EXT4 的块映射错误不会被放大为用户态段错误。

```
ext4_fread(file, buf, size):
    file_size = inode_size(file)
    size = min(size, file_size - file_pos)
    for each logical block in [file_pos, file_pos + size):
        physical = inode_extent_lookup(logical)
        if physical == 0:
            memset(output_range, 0) // sparse hole
        else:
            read physical block range // bcache/blockdev
            advance file position and output
    return bytes actually read
```

6.3.4 目录遍历

`vfs_ext4_getdents` 循环调用 `ext4_dir_entry_next` 获取下一个目录项，跳过 `.` 和 `..`，将 `lwext4` 目录项转换为 `linux_dirent64` 格式，计算对齐后的 `reclen` 确保不超出用户缓冲区，最后返回已写入的字节数。

6.3.5 元数据访问

统计信息函数 `vfs_ext4_stat`、`vfs_ext4_fstat`、`vfs_ext4_statx` 均通过 `ext4_raw_inode_fill` 读取原始 inode 和 superblock，然后逐字段填充 `struct kstat` 的模式、大小、时间戳（含纳秒部分从 `extra` 字段提取）。

6.3.6 删除操作中的 inode 引用协调

`vfs_ext4_unlinkat` 需要显式协调父目录和子节点的 inode 引用：获取挂载点，通过 `ext4_raw_inode_fill` 查找两者的 inode 号，获取 `inode_ref` 并加锁，通过 `strrchr` 提取子节点名称，调用 `_ext4_unlink` 完成底层删除，最后释放两个 `inode_ref`。这一流程说明目录项删除被视为需要在适配层明确协调的元数据操作。

6.4 块缓存层

块缓存层（`kernel/fs/bio.c`）为文件系统提供统一的磁盘缓冲抽象，维护一个双向链表按最近使用顺序排序。

6.4.1 核心数据结构

```
struct {
    struct spinlock lock; // 保护缓存元数据
```

```

    struct buf buf[NBUF]; // 缓冲块数组
    struct buf head; // 链表头节点 (head.next 为最近使用)
} bcache;

```

每个 `struct buf` 包含设备号、块号、有效位、磁盘持有标记、引用计数、睡眠锁和数据区 (`Bsize` 字节)。

6.4.2 四个核心接口

函数	行为
<code>bget(dev, blockno)</code>	查找缓存命中；未命中则回收最久未使用的空闲块
<code>bread(dev, blockno)</code>	<code>bget</code> 后检查 <code>valid</code> ；无效时调用底层驱动读入
<code>bwrite(b)</code>	将缓冲块内容写回磁盘
<code>brelse(b)</code>	释放睡眠锁，将块移到 LRU 链表头部

6.4.3 bget 的 LRU 回收策略

`bget` 从头（最近使用）向尾遍历已缓存块，若命中则增加 `refcnt`，返回锁定缓冲；若未命中，从尾（最久未使用）向前遍历，寻找 `refcnt == 0` 的空闲块，重新初始化后返回；若所有块均被引用，触发 `panic`。

```

struct buf* bread(uint dev, uint blockno) {
    b = bget(dev, blockno);
    if (!b->valid) {
#ifdef RISC_V
        virtio_rw(b, 0); // RISC-V: virtio-mmio
#else
        la_virtio_disk_rw(b, 0); // LoongArch: virtio-pci
#endif
        b->valid = 1;
    }
    return b;
}

```

6.4.4 blockdev.c 的 EXT4 适配

`kernel/fs/blockdev.c` 将 `bio.c` 的接口封装为 `lwext4` 需要的 `ext4_blockdev` 模式。读路径通过 `bread` 读取块后 `memmove` 到目标缓冲区再 `brelse`；写路径类似，使用 `bget + memmove + bwrite + brelse` 的顺序：

```

static int blockdev_read(struct ext4_blockdev *bdev,

```

```
void *buf, uint64_t blk_id, uint32_t blk_cnt)
{
    for (int i = 0; i < blk_cnt; i++) {
        struct buf *b = bread(0, blk_id + i);
        memmove(buf + i*BSIZE, b->data, BSIZE);
        brelse(b);
    }
    return EOK;
}
```

这样，上层 EXT4 语义请求经过 EXT4 语义 -> blockdev 适配 -> 操作系统缓冲缓存 -> VirtIO 驱动四层到达物理设备。

6.5 管道实现

管道实现位于 `kernel/fs/pipe.c`，使用环形缓冲区和累计计数管理读写位置。

```
struct pipe {
    struct spinlock lock;
    char data[PIPESIZE]; // 512 字节固定缓冲区
    uint nread; // 累计读取字节数
    uint nwrite; // 累计写入字节数
    int readopen; // 读端是否打开
    int writeopen; // 写端是否打开
};
```

缓冲区满条件为 `nwrite == nread + PIPESIZE`，避免了模运算和环绕处理。

写路径 (`pipewrite`)：获取管道锁后循环写入，缓冲区满时唤醒读端并在 `&nwrite` 上睡眠；若读端已关闭或进程被 `kill`，返回 `-1`。每次从用户空间拷贝 1 字节写入 `data[nwrite++ % PIPESIZE]`，完成后唤醒所有等待的读端。

读路径 (`piperead`)：获取管道锁，若无数据且写端仍打开，在 `&nread` 上睡眠。逐字节从 `data[nread++ % PIPESIZE]` 拷贝到用户空间，完成后唤醒写端。

6.6 BusyBox 虚拟文件

为支撑 BusyBox 和兼容测例的运行，系统在真实 EXT4 之外引入了一套轻量级虚拟文件机制。

6.6.1 虚拟文件列表

路径	内容类型	说明
<code>/proc/meminfo</code>	固定字符串	内存总量，分架构（RISC-V: 1GB / LA: 400MB）
<code>/proc/mounts</code>	固定字符串	两个挂载项（ <code>rootfs</code> + <code>proc</code> ）
<code>/proc/cpuinfo</code>	固定字符串	CPU 信息，分架构描述

/proc/config.gz	空字符串	兼容性占位
/proc/self/maps	动态生成	当前 ELF、栈和 VMA 的映射及权限
/proc/self/exe	动态路径	当前进程实际执行文件路径
/proc/self/status	动态生成	含 PID、PPID、UID 等
/proc/<pid>/stat	动态生成	按 PID 查询进程状态
/proc/sys/kernel/pid_max	固定字符串	"32768"
/proc/sys/fs/pipe-user-pages-soft	固定字符串	"16384"
/dev/misc/rtc	空字符串	RTC 占位
/etc/passwd	固定字符串	root 和 nobody 两个用户
/etc/group	固定字符串	root 和 nogroup 两个组
/proc/self/oom_score_adj	动态 OOM 值	可读可写
/proc/<pid>/oom_score_adj	动态 OOM 值	按 PID 读写

表 6.6: BusyBox 虚拟文件列表

6.6.2 按路径分发

busybox_virtual_content (kernel/fs/file.c) 通过字符串比较实现路径分发表。is_busybox_virtual_path 通过 get_fs_from_path 返回 NULL 且路径匹配虚拟文件前缀来判断。目录类型 (/proc、/proc/self、/etc) 在 sys_openat 中也进行特殊处理。

6.7 execve 装载器

程序装载由 sys_execve (kernel/syscall.c) 和 exec (kernel/exec.c) 组成。

6.7.1 执行路径与软链接解析

exec() 在读取 ELF 头部之前调用 resolve_exec_path() 规范化目标路径。该函数先将相对路径转换为相对于当前工作目录的绝对路径，再逐层读取软链接目标：绝对目标从根目录重新开始，相对目标则相对于当前软链接所在目录拼接。每一层都重新检查文件类型，最多解析 EXEC_MAX_SYMLINKS 层；超过上限或无法继续读取时终止装载，避免软链接循环把普通路径解析变成无限过程。

```

resolve_exec_path(path, resolved):
    current = absolute_path(path, cwd)
    for depth = 0 .. EXEC_MAX_SYMLINKS - 1:
        st = stat(current)
        if st is not symbolic link:
            return current
        target = readlink(current)
        if target starts with '/':
            current = target
        else:
            current = absolute_path(target, parent(current))
    return error

```

因此动态库或解释器的版本软链接会在进入 ELF 装载器前解析到真实文件，而不是

把软链接文本误当成 ELF 内容。典型路径为：libatomic.so.1 → 版本软链接 → 真实 ELF 文件 → 映射与动态链接器读取。

6.7.2 执行流程

`sys_execve()` 负责从用户空间复制路径、参数和环境变量，并完成兼容层预处理；随后调用 `exec()` 重新构造当前进程的用户地址空间，装载 ELF 或脚本解释器、建立用户栈并写入辅助向量，最后将控制权切换到新的用户入口。该过程保持进程身份不变，但以新的代码段、数据段和栈布局替换原有执行映像。

6.7.3 PT_LOAD 多段统一装载

主 ELF 不再把每个 PT_LOAD 段当作互不相关的连续内存区间，而是由 `elf_load_segments()` 先建立整个映像的页面集合，再复制文件内容并统一确定权限。这样多个段共享同一页时不会重复分配或覆盖另一段的数据，`p_filesz` 与 `p_memsz` 的差值也自然表现为已经清零的 BSS 区域。

其关键路径分为三个阶段：遍历全部 PT_LOAD，按段的内存范围逐页分配并清零；仅将每个段的 `p_filesz` 字节按文件偏移复制到页面中；再次遍历所有段，先清除临时的可写权限，再将每个段的最终权限合并到其覆盖页面，最后刷新 TLB。RISC-V 的可写页补充可读权限，两个架构的可写页均补充 PTE_D，从而使 ELF 段权限、共享页权限和架构页表约束保持一致。

```
elf_load_segments(pt, inode, ehdr, load_bias):
    for each PT_LOAD:
        check filesz <= memsz and address overflow
        for page in [vaddr, vaddr + memsz):
            if page is not mapped:
                allocate page, memset(page, 0)
                map page as PTE_R | PTE_W | PTE_U
            copy exactly filesz bytes at file offset p_offset

    for each PT_LOAD:
        clear PTE_R | PTE_W | PTE_X on covered pages
        OR the segment permission into every covered page
        add PTE_D when the page is writable
    sfence_vma()
```

装载完成后返回程序映像的低地址和页对齐后的高地址，供进程记录 `virt_addr/sz` 并继续安排动态映射。该算法同时处理段间空洞、共享页、BSS 和非零 PIE 基址，避免旧式按段扩展地址空间时出现的页重复映射与权限冲突。

6.7.4 PIE 支持

对于 ET_DYN 类型的 ELF (PIE 可执行文件), 装载器在 `load_bias = 0x10000` 的基础上映射:

```
if (ehdr.type == ELF_TYPE_DYN)
    load_bias = 0x10000UL;
uint64 seg_vaddr = load_bias + ph.vaddr;
```

选择 `0x10000` 作为基地址的设计理由: 保持低地址空页 (NULL 解引用可捕获), 同时使 PIE 程序的 `AT_*` 辅助向量值在动态链接器的合理范围内。

6.7.5 动态解释器处理

当 ELF 包含 PT_INTERP 程序头时, `exec` 进入动态链接路径。首先读取 `interp_name` 字符串 (如 `/lib/ld-linux-riscv64-lp64d.so.1`), 然后根据解释器名称匹配 `glibc` 或 `musl` 的对应路径, 读取解释器文件的 ELF 头部。主程序的完整映像已经由 `elf_load_segments()` 建立, 解释器随后由 `load_interpreter()` 通过 `mmap` 获取地址空间, 并按其全部 PT_LOAD 记录读取文件内容, 最终通过 `AT_BASE` 和入口地址交给动态链接器。

6.7.6 辅助向量构造

辅助向量 (`auxv`) 是内核向用户态动态链接器传递信息的关键机制。`exec` 通过 `alloc_aux` 构建:

辅助向量类型	值	用途
AT_HWCAP	0	硬件能力位图
AT_PAGESZ	PGSIZE (4096)	页大小
AT_PHDR	<code>load_bias + ph.vaddr</code>	程序头表地址
AT_PHENT	<code>ehdr.phentsize</code>	每个程序头大小
AT_PHNUM	<code>ehdr.phnum</code>	程序头数量
AT_BASE	<code>interp_start_addr</code>	动态链接器基址
AT_ENTRY	<code>load_bias + ehdr.entry</code>	程序入口点
AT_UID	0	用户 ID
AT_EUID	0	有效用户 ID
AT_GID	0	组 ID
AT_EGID	0	有效组 ID
AT_SECURE	0	安全模式标志
AT_RANDOM	栈上的 16 字节随机数地址	ASLR 熵来源
AT_EXECPN	用户栈中的实际执行路径字符串地址	动态链接器识别执行文件
AT_FLAGS	0	标志位
AT_NULL	0	结束标记

表 6.8: 用户栈上辅助向量列表

随机数写入用户栈后通过 `AT_RANDOM` 向动态链接器传递地址；实际执行路径则先作为字符串写入栈，再通过 `AT_EXECFN` 传递其地址。这样 `argc`、`argv`、`envp` 和 `auxv` 在同一套 64 位指针布局中连续构造，解释器可以按 Linux 进程启动约定读取。

6.7.7 脚本兼容与重定向

脚本识别不再把文件名后缀作为唯一条件。`is_sh_script()` 对常见的 `.sh` 路径保留快速判断，同时读取文件开头的两个字节识别 `#!`；因此没有后缀的可执行脚本也能进入解释器路径。

```
is_sh_script(path):
    if path ends with ".sh":
        return true // compatibility fast path
    magic = read(path, offset=0, size=2)
    return magic == "#!"

if is_sh_script(path):
    argv' = ["busybox", "sh", path, original_argv[1..]]
    exec_path = "/musl/busybox"
    exec(exec_path, argv', env)
```

脚本参数会保留在解释器参数之后，最终形成 `busybox sh [script] [原参数]`，避免把脚本正文或原始 `argv[0]` 错当作 ELF 路径。

重定向解析在 `argv` 中扫描 `>` 和 `>>` 符号。当在入口切换前检测到重定向时，`exec` 关闭当前的标准输出并调用 `vfs_ext4_open` 以 `O_WRONLY` 或 `O_WRONLY | O_APPEND` 模式打开目标文件。

6.8 本章小结

当前文件系统以 `struct file` 统一文件对象承接系统调用，以 `get_absolute_path` 和 `get_fs_from_path` 组织命名空间，以 `vfs_ext4.c` 对接 `lwext4`，以 `bio.c` 和 `blockdev.c` 贯通到底层块设备驱动。EXT4 读取路径能够区分有效块、稀疏空洞和文件尾部，避免文件系统数据错误传递到 ELF 和动态链接器。`execve` 装载器则完成软链接解析、`PT_LOAD` 多段页面和权限的统一处理、动态解释器选择、`shebang` 参数重组以及 `argc/argv/envp/auxv` 用户栈构造，使文件接口最终转化为稳定的程序加载能力。

第七章 驱动层设计与实现

7.1 概述

驱动层将硬件事件转化为内核可消费的接口。不同体系结构在寄存器布局、总线形式和中断入口上存在差异，但驱动层需要向文件系统和调度器提供稳定的抽象。当前驱动层组织为两条后端：RISC-V 侧以 VirtIO MMIO 驱动块设备，LoongArch 侧以 PCI/virtio-pci 驱动块设备；控制台、定时器和中断分发作为公共支撑层。块设备后端还必须明确 CPU 缓存、DMA 缓冲区和设备队列之间的所有权，随后通过统一的 `bio.c/blockdev.c` 向 EXT4 提供块读写接口。

驱动层各模块的依赖关系如图所示：

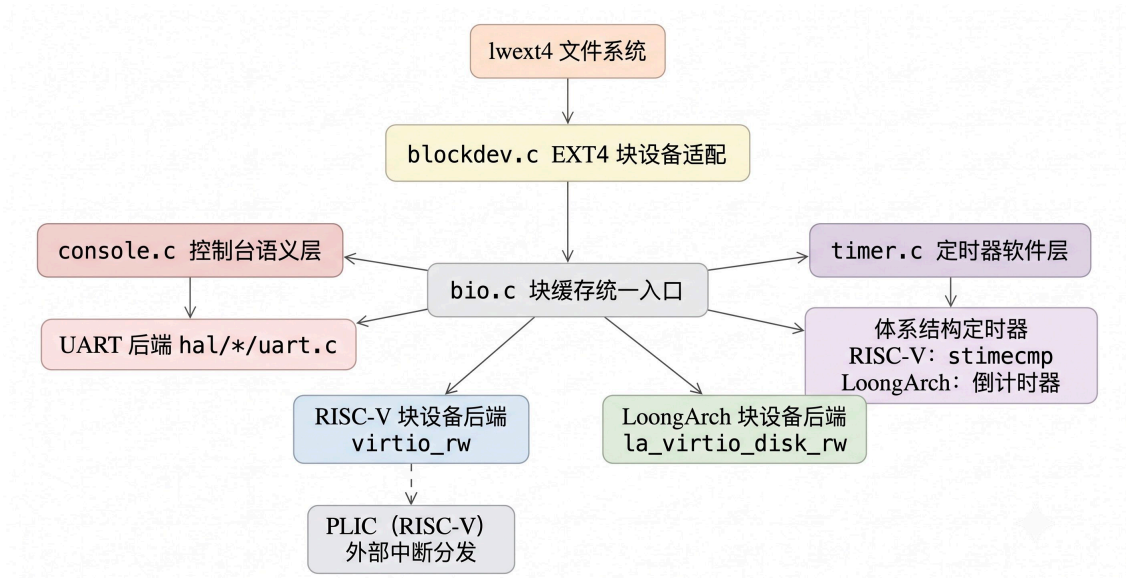


图 7.1: 驱动层模块依赖关系

7.2 控制台子系统

7.2.1 读写分离架构

控制台层（`kernel/console.c`）将字符输入输出的语义与硬件访问分离。核心接口分为三条路径：

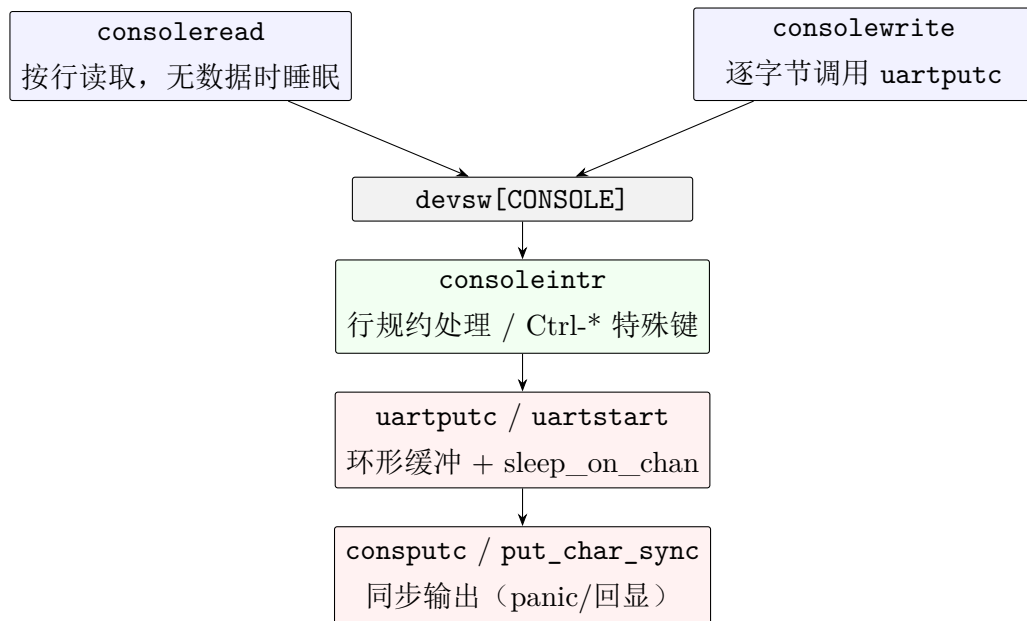


图 7.2: 控制台的读写分离架构

7.2.2 行规约处理

`consoleintr` 是控制台输入的统一入口，在持有 `cons.lock` 的条件下处理特殊输入：Ctrl-P (0x10) 触发 `procdump()`，Ctrl-U (0x15) 清除整行，退格 (Ctrl-H) 删除最后一个字符，回车 (\r) 规整为 \n，Ctrl-D (0x04) 触发文件结束语义；普通字符写入编辑区，遇到换行或缓冲填满时推进写入边界并唤醒阻塞的读者。

控制台输入缓冲区用三个指针维护：

```

struct {
    struct spinlock lock;
    char buf[INPUT_BUF_SIZE]; // 128 字节
    uint r; // 读取位置（已提交给读者的数据）
    uint w; // 写入边界（已确认的可读数据）
    uint e; // 编辑位置（当前行编辑位置）
} cons;
  
```

7.2.3 UART 发送环形缓冲

`uartputc` 和 `uartstart` (`hal/*/uart.c`) 构成写路径的协作对。`uartputc` 在环形缓冲满时通过 `sleep_on_chan(&uart_tx_r)` 睡眠；`uartstart` 从缓冲区取出字符写入 UART 数据寄存器，缓冲变空时唤醒等待的写者。`put_char_sync` 用于回显和 `panic` 等同步输出场景，轮询 UART 线路状态直到发送器空闲。

7.2.4 字符设备注册

`chardev_init` 完成控制台设备节点注册：初始化 `cons.lock`，调用 `uart_init()` 初始化 UART 硬件，将 `consoleread` 和 `consolewrite` 绑定到 `devsw[CONSOLE]`，注册

/dev/null 和 /dev/zero。

7.3 定时器

7.3.1 双架构定时器初始化对比

RISC-V 和 LoongArch 采用不同的硬件定时器机制：

特性	RISC-V	LoongArch
定时器类型	比较寄存器 stimecmp	倒计时计数器 (countdown)
中断触发条件	<code>time >= stimecmp</code>	计数减到 0
重装方式	每次中断后软件写 stimecmp	硬件自动重装 (periodic 位)
SBI 路径	<code>set_timer(r_time() + INTERVAL)</code>	不支持
非 SBI 路径	<code>w_stimecmp(r_time() + INTERVAL)</code>	<code>w_csr_tcfg(TCFG)</code>

表 7.2: RISC-V 与 LoongArch 定时器机制对比

RISC-V 的 `timer_init` (`kernel/timer.c`) 初始化 `tickslock`、清零 `ticks`，并通过 `read_rtc_seconds()` 读取 RTC 寄存器获得 `boot_time`。LoongArch 的 `countdown_timer_init` 配置 TCFG 寄存器使能周期性倒计时：

```
void countdown_timer_init(void)
{
    prcfg1_val = r_csr_prcfg1();
    timerbits = FIELD_GET(prcfg1_val, TIMERBITS) + 1;
    // 构造 TCFG 值：使能 + 周期重装 + 初始值
    tcfg_val = FIELD_WRITE(1, TCFG_EN) |
                FIELD_WRITE(1, TCFG_PERIODIC) |
                FIELD_WRITE(INTERVAL, TCFG_INITVAL(timerbits));
    w_csr_tcfg(tcfg_val);
}
```

7.3.2 事件链：timer_tick 到 SIGALRM

`timer_tick` 是定时器中断的公共处理逻辑：递增全局 `ticks`，唤醒等待时钟推进的睡眠者；遍历进程池对每个进程调用 `refresh_process_timer`；RISC-V 上预约下一次中断，LoongArch 上倒计时器自动重装。

`refresh_process_timer` 将进程级 interval timer 绑定到时间轴：

```
static void refresh_process_timer(proc_t *p, uint64 now)
```

```

{
    if (!p->timer_active || now < p->alarm_ticks)
        return;
    // 超时: 设置 SIGALRM 信号
    p->sig_pending.__val[0] |= (1UL << SIGALRM);
    interval = timeval_to_clocks(&p->itimer.it_interval);
    if (interval == 0) {
        // 一次性定时器
        p->timer_active = 0;
    } else {
        // 周期性定时器: 推进 alarm_ticks
        do {
            p->alarm_ticks += interval;
        } while (p->alarm_ticks <= now);
    }
}
}

```

7.4 RISC-V PLIC

平台级中断控制器 (PLIC) 用于管理和分发 RISC-V 的外部中断。实现位于 `hsai/plic.c`。 `plicinit()` 为 UART0 和 VIRTIO0 设置优先级, `plicinithart()` 使能 S 态中断源, `plic_claim()` 获取待处理中断 IRQ, `plic_complete()` 通知 PLIC 处理完成。

在 Trap 层 (`hsai/hsai_trap.c`) 的 `devintr()` 中, PLIC 用于设备中断分类:

```

int devintr() { // RISC-V 路径
    scause = r_scause();
    if (scause == SUPERVISOR_EXTERNAL_INTERRUPT) {
        irq = plic_claim();
        if (irq == VIRTIO0_IRQ)
            virtio_disk_intr(); // 回收块设备请求
        else if (irq == UART0_IRQ)
            uartintr(); // 处理串口输入
        plic_complete(irq);
        return 1;
    } else if (scause == SUPERVISOR_TIMER_INTERRUPT) {
        timer_tick();
        return 2;
    }
    return 0;
}

```

7.5 RISC-V VirtIO MMIO 块设备

RISC-V 块设备驱动位于 `kernel/driver/riscv/virt.c`, 采用 MMIO 方式与 VirtIO 设备通信。

7.5.1 virtio_disk_init 状态机

初始化遵循 VirtIO 协议的标准状态机：

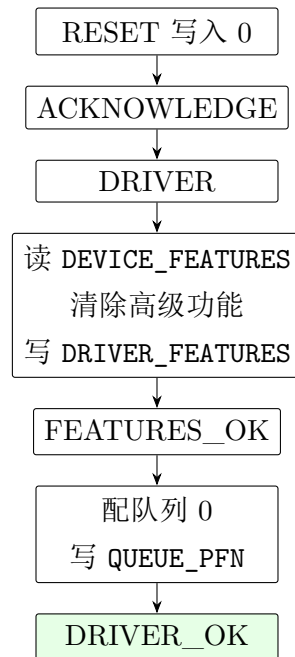


图 7.3: virtio_disk_init 的状态机

特性协商中显式关闭的能力包括：VIRTIO_BLK_F_RO（只读）、VIRTIO_BLK_F SCSI、VIRTIO_BLK_F_CONFIG_WCE（写缓存）、VIRTIO_BLK_F_MQ（多队列）、VIRTIO_F_ANY_LAYOUT、VIRTIO_RING_F_EVENT_IDX 和 VIRTIO_RING_F_INDIRECT_DESC。这表示当前实现选择了一条保守但容易验证的 VirtIO 块设备子集。

7.5.2 三描述符链与请求提交

virtio_rw 对每次块请求构造三个描述符:

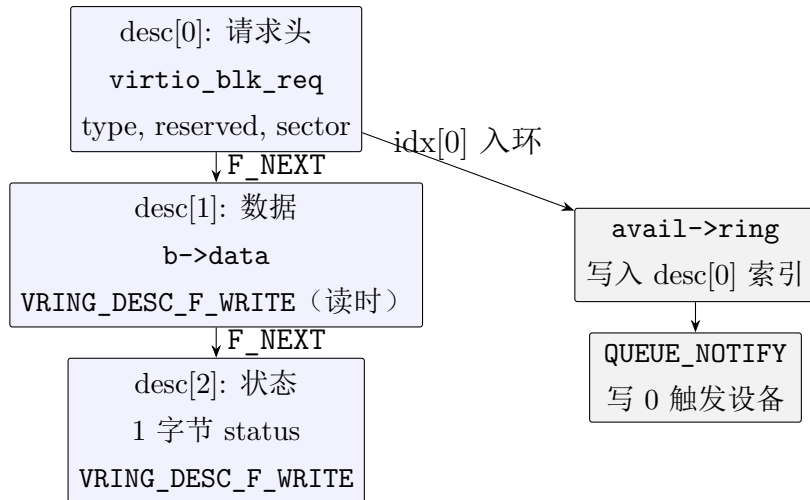


图 7.4: virtio_rw 的三描述符链

请求提交后通过忙等待同步:

```

b->disk = 1; // 标记 buf 为设备持有
disk.avail->ring[disk.avail->idx % NUM] = idx[0];
__sync_synchronize();
disk.avail->idx += 1; // 通知设备有可用描述符
__sync_synchronize();
*R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0; // 触发设备处理

while (b->disk == 1) { } // 忙等请求完成
  
```

RISC-V 路径直接将 `b->data` 作为设备数据描述符, 完成请求后由 `virtio_disk_intr()` 读取状态字节并清除 `b->disk`。因此其数据所有权由块缓存页直接转移给设备, 再由中断回收; LoongArch 则使用独立 DMA 缓冲区, 具体差异在后文统一说明。

7.5.3 中断回收

`virtio_disk_intr` 被 Trap 层的 `devintr` 在识别到 `VIRTIO0_IRQ` 后调用: 读 `INTERRUPT_STATUS` 并写入 `INTERRUPT_ACK` 确认中断, 然后循环处理 `used` 环中所有新完成项, 通过 `info[id].b` 获取对应的 struct buf 并将 `b->disk` 清零。

7.5.4 描述符表内存布局

```

// pages 是两页对齐的内存
disk.desc = (struct virtq_desc *)pages; // 描述符表
disk.avail = (struct virtq_avail *) (pages + // 可用环
    NUM * sizeof(struct virtq_desc));
  
```

```
disk.used = (struct virtq_used *) (pages + PGSIZE); // 已用环
```

三个队列在共享内存中的布局以页边界分割：第一页包含描述符表和可用环，第二页包含已用环。设备通过 QUEUE_PFN 寄存器和页框号访问这片共享内存。

7.6 LoongArch PCI 与 VirtIO-PCI 块设备

LoongArch 块设备后端由三个文件构成：kernel/driver/loongarch/pci.c、kernel/driver/loongarch/virtio_pci.c 和 kernel/driver/loongarch/virtio_disk.c。

7.6.1 DMA 缓冲区与非缓存访问

LoongArch 的设备 DMA 直接访问物理内存，而 CPU 对同一物理内存的普通访问可能经过缓存。驱动因此把设备共享区域分为两类访问视图：队列、请求头、状态字节和 DMA 缓冲区统一通过 DMWIN1 的非缓存别名访问；块缓存中的 b->data 保持为内核缓存数据，不直接交给设备。写请求先从 b->data 复制到独立的 dma_buf，读请求完成后从 dma_buf 显式复制回 b->data，从接口上隔离 CPU 缓存和 DMA 所有权。

提交队列前后的 __sync_synchronize() 保证描述符内容、可用环索引和设备通知按照正确顺序对设备可见；完成阶段先观察非缓存状态字节，再读取 used 环和数据缓冲区。disk.vdisk_lock 将单个 DMA 缓冲区、描述符元数据和完成回收串行化，避免定时器驱动的重入覆盖尚未回收的请求。

```
la_virtio_disk_rw(b, write):
    acquire(disk.vdisk_lock)
    allocate three descriptors
    if write:
        memcpy(uncached(dma_buf), b->data, BSIZE)
    desc[0] = request header
    desc[1] = uncached(dma_buf), device read/write direction
    desc[2] = uncached(status), initial value 0xff
    memory_barrier()
    publish avail entry and notify device
    memory_barrier()

    while status == 0xff:
        poll completion status
    validate used element and status
    if not write:
        memcpy(b->data, uncached(dma_buf), BSIZE)
    release(disk.vdisk_lock)
```

7.6.2 PCI 设备枚举

pci_device_probe 在 ECAM (Enhanced Configuration Access Mechanism) 空间中遍历 bus、device 和 function:

```
uint64 pci_device_probe(uint16 vendor_id, uint16 device_id)
{
    for (bus = 0; bus < 256; bus++)
        for (dev = 0; dev < 32; dev++)
            for (func = 0; func < 8; func++) {
                uint64 off = (bus << 16) | (dev << 11) | (func << 8);
                uint32 id = pci_config_read32(off + PCI_VENDOR_ID);
                if (id == (vendor_id | (device_id << 16))) {
                    // 匹配成功: 记录 bus/dev/func
                    // 写 PCI_COMMAND 开启 I/O、memory、bus master
                    return off; // 返回 ECAM 偏移
                }
            }
}
}
```

7.6.3 Capability 解析

virtio_pci_read_caps 从配置空间偏移 0x34 读取 capability 链表, 遍历所有 vendor-specific capability:

```
int virtio_pci_read_caps(virtio_pci_hw_t *hw, uint64 pci_base,
                        trap_handler_fn *msix_isr)
{
    cap_ptr = pci_config_read8(pci_base + PCI_CAPABILITY_LIST);
    while (cap_ptr) {
        cap_id = pci_config_read8(pci_base + cap_ptr);
        cap_len = pci_config_read8(pci_base + cap_ptr + 1);
        if (cap_id == PCI_CAP_ID_VENDOR) {
            cfg_type = pci_config_read8(pci_base + cap_ptr + 3);
            bar = pci_config_read8(pci_base + cap_ptr + 4);
            offset = pci_config_read32(pci_base + cap_ptr + 8);
            // 根据 cfg_type 分别保存:
            // common_cfg, notify_cfg, device_cfg, isr_cfg
        }
        cap_ptr = pci_config_read8(pci_base + cap_ptr + 2);
    }
    // 四类能力缺一即判定为非现代 virtio-pci 设备
}
```

7.6.4 轮询完成路径

la_virtio_disk_rw (kernel/driver/loongarch/virtio_disk.c) 与 RISC-V 版本的差异集中在设备访问和完成回收方式，而不是上层块请求语义：LoongArch 通过 PCI/VirtIO-PCI 配置队列，使用 DMWIN1 非缓存别名访问共享区域，以独立 DMA 缓冲区承接数据，并在请求内轮询状态字节完成同步。

提交时先写入三描述符链，将请求头、DMA 数据缓冲区和状态字节分别交给设备；然后通过内存屏障发布 avail 环索引并通知队列。完成时不能用 `used->id == 0` 作为“尚未完成”的判断，因为首个请求的合法描述符索引也可能为 0。驱动把状态字节初始化为 `0xff`，设备完成后写入结果码，轮询条件因此变为“状态仍为 `0xff`”；确认完成后再读取 `used` 环元素，校验其描述符链首索引与请求一致，最后检查状态码、回拷读数据并释放描述符。

```
acquire(&disk.vdisk_lock)
// desc[0]: request, desc[1]: dma_buf, desc[2]: status
status = 0xff
avail[2 + (avail[1] % NUM)] = first_desc
__sync_synchronize()
avail[1] += 1
__sync_synchronize()
virtio_pci_set_queue_notify(&gs_virtio_blk_hw, 0)

while (*status == 0xff) { } // 状态字节完成同步
__sync_synchronize()
id = used->elems[used_idx % NUM].id
assert(id == first_desc && *status == 0)
if (!write) memcpy(b->data, uncached(dma_buf), BSIZE)
used_idx += 1
free_chain(id)
release(&disk.vdisk_lock)
```

7.6.5 两种架构的差异对比

特性	RISC-V VirtIO MMIO	LoongArch VirtIO-PCI
设备接口	MMIO 寄存器	PCI 配置空间 + BAR
队列通信	共享内存 (QUEUE_PFN)	set_queue_addr2
通知方式	写 QUEUE_NOTIFY 寄存器	set_queue_notify
完成回收	独立中断回调 virtio_disk_intr	状态字节轮询后校验 used->elems[].id
请求同步	while(b->disk==1) 忙等	while(*status==0xff) 忙等

数据所有权	直接使用 b->data	独立 dma_buf，完成后显式回拷
共享区域访问	普通共享内存	DMWIN1 非缓存别名与内存屏障
并发控制	单队列串行假设	vdisk_lock 保护请求生命周期
特性协商	MMIO 状态寄存器	PCI 能力寄存器

表 7.4: RISC-V 与 LoongArch 块设备驱动对比

7.7 驱动与上层接口

kernel/fs/bio.c 和 kernel/fs/blockdev.c 是驱动层与文件系统的衔接面：前者通过条件编译分发 bread/bwrite，后者将其封装为 lwext4 的 ext4_blockdev 接口。上层只看到统一的 struct buf 和块设备回调，不感知 MMIO、中断、PCI、轮询或独立 DMA 缓冲区。

```
blockdev_read(blk_id, blk_cnt):
    for each block i:
        b = bread(blk_id + i)
        copy b->data to the lwext4 buffer
        brelse(b)
```

I/O 主路径为：lwext4 → blockdev.c → bio.c → RISC-V VirtIO-MMIO 或 LoongArch VirtIO-PCI；后端完成中断回收或状态轮询后，将数据和 valid/disk 状态交还给块缓存，EXT4 和 inode 层无需增加设备分支。

7.8 本章小结

驱动层包含三个功能方向：字符设备以 kernel/console.c 为语义中心，以 hal/*/uart.c 为硬件后端，通过 devsw[] 接入 VFS；定时器通过 kernel/timer.c 将硬件时钟中断转化为进程定时器和 SIGALRM 信号，经 yield() 与调度器衔接；块设备在 RISC-V 上采用 VirtIO-MMIO 与 PLIC 中断回收，在 LoongArch 上采用 PCI/VirtIO-PCI、DMWIN1 非缓存访问、独立 DMA 缓冲区和状态字节轮询，最终统一汇入 kernel/fs/bio.c 与 kernel/fs/blockdev.c。这一分层同时明确了 CPU 缓存、DMA 数据和设备队列的所有权边界。

第八章 总结与展望

8.1 设计总结

内核以 `proc_t`、`thread_t`、`cpu_t` 分离进程资源、线程现场和 CPU 调度现场，以 Buddy、Slab、页表和 VMA 构成内存管理层次，并通过独立的 `vma_lock`、完整 VMA 区间操作和按实际权限设置 PTE 支撑动态映射。HAL 与 HSAI 将 RISC-V 和 LoongArch 的启动、Trap、中断、页表及用户态返回差异封装在架构边界内，其中多 hart 启动同步、LoongArch PME、LSX/LASX 和页面权限得到统一处理。系统调用层通过结构转换、clone 参数归一化和 TCGETS 长度控制实现 Linux ABI 兼容，并动态生成 `/proc/self/maps` 和真实执行路径；文件系统与装载器则结合软链接解析、脚本识别、统一 PT_LOAD 装载、动态解释器、用户栈辅助信息和稀疏文件补零完成用户程序启动。驱动层在 RISC-V 上使用 VirtIO-MMIO，在 LoongArch 上使用 PCI/VirtIO-PCI，通过非缓存访问、独立 DMA 缓冲区、内存屏障和状态字节明确设备所有权，最终经 `kernel/fs/bio.c` 与 `kernel/fs/blockdev.c` 向 EXT4 提供统一块 I/O。

这些设计共同形成从架构入口、任务调度、地址空间和系统调用，到文件系统、程序装载和设备 I/O 的完整内核路径。各层通过稳定接口连接，架构差异和设备细节被限制在 HAL、HSAI 及驱动后端边界内。

8.2 后续优化方向

后续工作针对当前实现的剩余边界、语义完整性和并发可扩展性展开。已经完成的线程退出、基础 VMA 并发、动态虚拟文件、ELF 多段装载和 DMA 所有权不再作为“尚未实现”的计划；仍需完善的内容如下表所示。

表 8.1: 后续优化方向

方向	当前状态	计划改进
调度与信号语义	<code>scheduler()</code> 、 <code>futex_wait()</code> 、线程退出回收、线程定向信号和可中断阻塞已形成运行期控制流	补充完整 signal frame 与用户态处理函数递送，完善异常路径恢复、调度公平性；后续规划跨 CPU 唤醒、线程迁移和负载均衡
内存权限与并发	Buddy 防御、页表权限和带 <code>vma_lock</code> 的 VMA 区间操作已形成一致性基础	完善多 CPU 场景下的 TLB shutdown、共享页回收和地址空间同步，并继续进行并发 <code>mmap/munmap/mprotect</code> 回归验证

文件系统标准化	EXT4 主路径、稀疏文件读取、动态虚拟文件和 ELF/动态链接器装载已支撑 BusyBox/LTP 等场景	完善挂载语义、路径规则和缓存热路径，补齐 inotify、epoll 等运行时接口，并扩大文件系统与动态链接器回归测试
驱动与性能	RISC-V VirtIO-MMIO 和 LoongArch PCI/VirtIO-PCI 已统一接入块 I/O，DMA 缓冲区、非缓存访问和完成一致性边界明确	完善中断完成路径、错误恢复和新设备类型接入；后续规划并发块 I/O、设备队列扩展和多 CPU I/O 回归验证
